Anbindung der Google V8 Javascriptengine an ein Multi-Agent-SCADA-System

Daniel Giritzer BSc.



MASTERARBEIT

eingereicht am
Fachhochschul-Masterstudiengang

Embedded-Systems-Design

in Hagenberg

im August 2021

Betreuung:

FH-Prof. Dipl.-Ing (FH) Franz Wiesinger

© Copyright 2021 Daniel Giritzer BSc.

Alle Rechte vorbehalten

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Hagenberg, am 23. August 2021

Daniel Giritzer BSc.

Inhaltsverzeichnis

Erklärung iv

Kurzfassung viii

Abstract ix

1 Einleitung 1
- 1.1 Motivation für den Einsatz von SCADA-Systemen . . . 1
- 1.2 Probleme bei der Systemintegration von SCADA-Systemen . . . 2
- 1.3 Zielsetzung . . . 3
- 1.4 Gliederung dieser Arbeit . . . 3
- 1.5 Codebeispiele . . . 3
  - 1.5.1 Windows 10 . . . 3
  - 1.5.2 SUSE Linux Enterprise 12.5 . . . 4

2 Multi-Agent-SCADA-Systeme 5
- 2.1 SCADA - Supervisory Control And Data Acquisition . . . 5
- 2.2 Agent . . . 6
- 2.3 Simatic Windows Control Center Open Architecture . . . 6
  - 2.3.1 Basis Agenten und Konzept . . . 7
  - 2.3.2 Projektaufbau und Hierarchie . . . 12
  - 2.3.3 C++-Programmierschnittstelle . . . 13
- 2.4 SiPlant als Erweiterung . . . 18

3 Google V8 Javascriptengine 20
- 3.1 Grundlagen . . . 20
  - 3.1.1 Sprachprozessoren . . . 20
  - 3.1.2 Frontend . . . 21
  - 3.1.3 Umsetzung in Javascriptengines . . . 22
- 3.2 Wahl der Javascriptengine . . . 22
  - 3.2.1 Fazit . . . 23
- 3.3 Google V8 Javascriptengine im Detail . . . 24
  - 3.3.1 Abstrakter Syntaxbaum . . . 24
  - 3.3.2 Interpreter . . . 25
  - 3.3.3 Just-In-Time Compiler . . . 26
- 3.4 C++-Programmierschnittstelle . . . 27

3.4.1 Isolates . . . 27
3.4.2 Contexts . . . 28
3.4.3 Speicherverwaltung . . . 29
3.4.4 Anpassen von Javascript-Ausführungsumgebungen über das Global-Objekt . . . 29
3.4.5 Das DebugInspector Protokoll . . . 30
3.4.6 Beispiel . . . 30

4 Anforderungen **35**
4.1 Zu unterstützende Plattformen . . . 35
4.2 Anwendungsfälle . . . 35
4.2.1 Entwicklung kundenspezifischer Anforderungen . . . 36
4.2.2 Skriptgestützte Massenparametrierung . . . 37
4.3 Qualitätsanforderungen . . . 38
4.3.1 Benutzbarkeit . . . 38
4.3.2 Wartbarkeit . . . 38
4.3.3 Zuverlässigkeit . . . 38
4.4 Einsatz in spezifischeren C++-API-Agenten . . . 39

5 Konzept **40**
5.1 Compiler . . . 40
5.2 Linken der Google V8 Engine . . . 40
5.2.1 Statisches Linken . . . 40
5.2.2 Dynamisches Linken . . . 41
5.2.3 Fazit . . . 41
5.3 Realisierungsart . . . 41
5.3.1 Dynamische Bibliothek . . . 41
5.3.2 Statische Bibliothek . . . 41
5.3.3 Ausführbare Datei . . . 41
5.3.4 Fazit . . . 41
5.4 Handbuch für Integratorinnen und Integratoren . . . 42
5.4.1 TypeDoc . . . 43
5.4.2 Fazit . . . 44
5.5 Konzeptioneller Beweis . . . 44

6 Entwicklung des Prototypen **48**
6.1 Übersetzten des Google V8 C++ Quellcodes . . . 48
6.1.1 Linux . . . 48
6.1.2 Windows . . . 49
6.2 Implementierung der Programmbibliothek . . . 49
6.2.1 Implementierung der Singleton-Klasse . . . 49
6.2.2 Implementierung der libV8-Klasse . . . 50
6.2.3 Implementierung der SIS_InspectorClient-Klasse . . . 51
6.2.4 Implementierung der SIS_InspectorFrontend-Klasse . . . 52
6.2.5 Implementierung der V8Context-Klasse . . . 52
6.2.6 Implementierung des ICustomJSCommands-Interface . . . 53
6.3 Implementierung des Agenten . . . 54

7 Praxiserfahrungen **56**
7.1 BMW Gruppe, Dingolfing Deutschland 56
7.2 Daimler AG, Factory 56 Sindelfingen Deutschland 56
7.3 Valmet, Salo Finnland 56
7.4 Geely, Ningbo China 57
7.5 Fazit 57
7.5.1 Nachteile 57

8 Zusammenfassung und Ausblick **58**
8.1 Zusammenfassung 58
8.2 Ausblick 59
8.2.1 Fremdfunktionsschnittstelle (FFI) 59
8.2.2 Fazit 61

A Google V8 Javascriptengine Buildskripte **62**
A.1 Linux 62
A.2 Windows 63

B Inhalt der CD-ROM/DVD **66**

C Ausgabe einer V8-Javascript-Exception **67**

D Klassendiagramm **68**

Quellenverzeichnis **72**
Literatur 72
Audiovisuelle Medien 73
Online-Quellen 73

Kurzfassung

Aufgrund der Unterschiede zwischen den einzelnen Anlagen der Automatisierungsindustrie und der kundenspezifischen Projektanforderungen ist die Anlagenintegration von Multi-Agent-SCADA (Supervisory Control And Data Acquisition) Systemen oft mit vielen unvorhersehbaren und komplexen Problemen verbunden. Um diese zu lösen, ist ein einfacher Weg zur Entwicklung und Anpassung vor Ort erforderlich.

Eine Möglichkeit, dies zu erreichen, besteht darin, dem Systemintegrator eine dynamisch erweiterbare Skriptsprache mit Debugging-Funktionen und einer Interaktionsmethode für den Fernzugriff auf das SCADA-System bereitzustellen. Javascript eignet sich für diesen Zweck, da online und offline Literatur dazu gefunden werden kann. Außerdem gibt es viele öffentlich verfügbare Komponenten und Bibliotheken, um die Sprache zu erweitern, und die meisten Interpreter verfügen über Debugging-Protokolle, die die Entwicklung vereinfachen.

Die folgende Arbeit beschreibt, wie die Google V8 Javascriptengine in das kommerzielle WinCC OA (Windows Control Center Open Architecture) Multi-Agent-SCADA System für Windows und Linux als Teil des Produktes *SiPlant* eingebettet wird. Dies geschieht durch die Implementierung eines dedizierten Agenten, der in der Lage ist, Javascript-Code unter Verwendung der von Google bereitgestellten Engine auszuführen.

Der neu entwickelte Agent erweitert die Skriptsprache um einen Satz von benutzerdefinierten Befehlen, die den Zugriff auf die Daten des gesamten Systems durch die Verwendung der ereignisbasierten Konzepte von WinCC OA ermöglichen.

Abschließend wird der erstellte Agent in der Praxis auf verschiedenen Plattformen und Projekten untersucht.

Abstract

Due to the difference of every plant in the automation industry and the varying customer-specific project requirements, plant integration of multi-agent SCADA (Supervisory Control And Data Acquisition) systems often comes along with many unforeseen and complex problems. To overcome these problems, a simple way for on-site engineering and customization is needed.

This can be achieved, by providing the system integrator a dynamically extendable and well-documented scripting language with advanced debugging capabilities and an interaction method to access the SCADA system remotely. Javascript fits this purpose, as there can be literature found online and offline. Also, there are many publicly available components and libraries to extend the language and most interpreters feature debugging protocols which help to ease the development.

The following thesis describes how the Google V8 Javascriptengine is embedded as a part of the *SiPlant* product into the commercial WinCC OA (Windows Control Center Open Architecture) Multi-Agent-SCADA System for Windows and Linux. This is done by implementing an dedicated agent which is capable of executing Javascript code by utilizing the formerly named engine developed by Google.

This custom agent extends the scripting language by a set of custom commands which allow accessing the data of the whole system by using WinCC OA's event-based concepts.

Finally, the created agent is examined in practice on different platforms and projects.

Kapitel 1

Einleitung

In diesem Kapitel wird die behandelte Problemstellung und Motivation der vorliegenden Arbeit dargestellt und eine Zielsetzung definiert. Außerdem wird auf die Gliederung dieser Arbeit eingegangen und der Inhalt der einzelnen Kapitel kurz erläutert.

1.1 Motivation für den Einsatz von SCADA-Systemen

Ein grundlegendes Problem bei der Digitalisierung von Fabriken ist, dass Cloudanwendungen, die Anlagen steuern und überwachen wollen, und Anlagen, wiederum deren Daten in der Cloud ablegen wollen. Um diesen Konflikt zu lösen, werden sogenannte SCADA-Systeme eingesetzt. SCADA ist ein Akronym und steht für Supervisory Control And Data Acquisition. Diese Systeme implementieren laut Automatisierungspyramide Ebene zwei und Teile von Ebene drei.

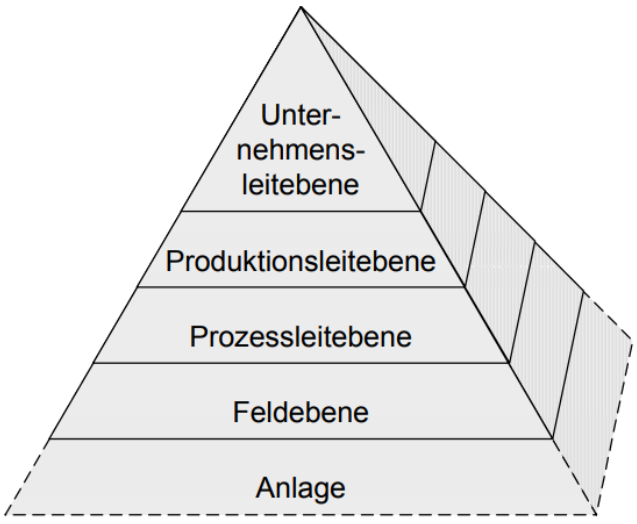


Abbildung 1.1: Diese Abbildung stellt die verschiedenen Ebenen der industriellen Fertigung mit Hilfe der Automatisierungspyramide dar [21]. Cloudanwendungen befinden sich in der Ebene vier, die auch *Unternehmensleitebene* genannt wird.

Die in Abbildung 1.1 gezeigten Ebenen können wie folgt zusammengefasst werden [14]:

- Ebene vier: Die *Unternehmensleitebene* umfasst Produktionsplanung und Bestellabwicklung.
- Ebene drei: Stellt die *Produktionsleitebene* dar, in der Produktionsdaten erfasst werden. Außerdem findet hier Qualitäts- und Materialmanagement statt.
- Ebene zwei: In der *Prozessleitebene* wird bedient, beobachtet, es werden Daten gesammelt und archiviert.
- Ebene eins: Steuert die Anlage über SPSen (Speicherprogrammierbaren Steuerungen), die mit dieser über Bussysteme verbunden sind. Daher wird diese Ebene auch *Steuerungsebene* oder *Feldebene* genannt. Weitere Teilnehmer sind unter anderem Frequenzumrichter und Industrie-PCs.
- Ebene null: Als *Anlage* wird die zu steuernde Maschine und die dazugehörige Sensorik bezeichnet.

Unter SCADA versteht man das Überwachen und Steuern technischer Prozesse durch Computersysteme [8]. Diese Systeme helfen bei der Überwachung von Anlagen und sind somit Wegbereiter, um Verfügbarkeit und Produktivität zu erhöhen. Am Siemensstandort in der Krausstraße in Linz befindet sich die Abteilung *DF FA SO SIS*. Der Abteilungsname beschreibt den Tätigkeitsbereich und steht als Abkürzung für Digital Factory, Factory Automation, Solution Operation und Shopfloor IT Solutions. Am Standort in Linz wird unter dieser Division an der Entwicklung von Lösungen für digitalisierte Fabriken gearbeitet. Ein Aufgabenbereich ist der Entwurf des webbasierenden Supervisory Control And Data Acquisition (SCADA) Systems *SiPlant* für die Automobilindustrie. Dieses Produkt kann als Erweiterung der Softwarelösung *Simatic Windows Control Center Open Architecture* (WinCC OA) der Siemens Tochtergesellschaft ETM Professional Control gesehen werden. WinCC OA eignet sich als Basissystem, da die Multi-Agent-Architektur ein einfaches Erweitern über eine C++-Programmierschnittstelle ermöglicht. Außerdem werden bereits viele Möglichkeiten zum akquirieren von Daten über proprietäre oder offene Feldbus Protokolle zur Verfügung gestellt. Als Beispiele können hier Profinet, MODBUS, S7Plus, OPC UA und BACNET angeführt werden [5]. Um plattformunabhängig moderne Visualisierungen für die Überwachung von Anlagen umsetzten zu können, wurde für *SiPlant* HTML5 als Technologie gewählt. Dies wurde mit Hilfe von eigens entwickelten WinCC OA Agenten umgesetzt.

1.2 Probleme bei der Systemintegration von SCADA-Systemen

Verschiedene Kundenstandards in den SPS-Programmen (Level 1 laut Automatisierungspyramide) und proprietäre Protokolle Richtung *Unternehmensleitebene* führen dazu, dass bei jeder Systemintegration individuelle Probleme gelöst werden müssen. Außerdem müssen Systemintegratorinnen und Systemintegratoren kundenspezifische Anwendungsfälle umsetzen. Hierbei bieten die Bordmittel von WinCC OA nur eingeschränkt Möglichkeiten.

1.3 Zielsetzung

Aufgrund der Unterschiede zwischen den einzelnen Anlagen der Automatisierungsindustrie und den verschiedenen kundenspezifischen Projektanforderungen ist die Integration des *SiPlant* Produktes mit vielen unvorhersehbaren und komplexen Problemen verbunden. Ziel dieser Arbeit ist es daher, die Google V8 Javascriptengine und das dazugehörige Debuginspector-Protokoll für das Produkt *SiPlant* an das Basissystem WinCC Open Architecture anzubinden, um damit den Systemintegratorinnen und Systemintegratoren ein Werkzeug für die Fehlersuche und zum Umsetzen von Kundenanforderungen mit Hilfe einer einfachen Skriptsprache zur Verfügung zu stellen.

1.4 Gliederung dieser Arbeit

Die vorliegende Arbeit ist in acht verschiedene Kapitel gegliedert. Kapitel 1 Umfasst die Motivation und Zielsetzung dieser Arbeit. In Kapitel 2 wird definiert, was unter dem Terminus Multi-Agent-SCADA-System verstanden wird. Danach wird die Umsetzung eines solchen anhand des Produktes *Simatic Windows Control Center Open Architecture* aufgezeigt und dessen C++-Programmierschnittstelle praktisch erklärt. Kapitel 3 zeigt, wie die Google V8 Javascriptengine funktioniert und wie sie von C++-Entwicklern in eigene Programme eingebettet werden kann. Das Kapitel 4 definiert den Funktionsumfang des in dieser Arbeit entwickelten Agenten anhand von Anwendungsfällen für Systemintegratorinnen und Systemintegratoren. Der konzeptuelle Aufbau des Agenten, das Zusammenspiel der einzelnen Komponenten und die Umsetzung der in Kapitel 4 festgelegten Anforderungen werden in Kapitel 5 beschrieben. Auf Implementierung mit Hilfe der festgelegten Konzepte aus Kapitel 5 und die Lösung aller aufkommenden Probleme wird in Kapitel 6 eingegangen. In Kapitel 7 werden die durch den Einsatz des in dieser Arbeit entwickelten Agenten gesammelten Erfahrungen aufgeführt. Das letzte Kapitel 8 umfasst die Zusammenfassung der Erkenntnisse dieser Arbeit und ein Fazit aus den Praxiserfahrungen. Darüber hinaus werden in einem Ausblick Vorschläge für Verbesserungen formuliert.

1.5 Codebeispiele

In dieser Arbeit werden verschiedene Konzepte und Programmierschnittstellen anhand von Codebeispielen erklärt. Diese Beispiele sind als vollständige Projekte der CD-ROM/DVD beigelegt und können auf 64 Bit Systemen mit Windows 10 oder SUSE Linux Enterprise 12.5 übersetzt werden. Dazu werden folgende Abhängigkeiten benötigt:

1.5.1 Windows 10

- Visual Studio 2019
- Visual Studio Toolset v140
- WinCC OA Version 3.16 inklusive C++-Programmierschnittstelle
- CMake mit mindestens Version 3.5
- Den SMF_3P Ordner von der CD-ROM/DVD

1.5.2 SUSE Linux Enterprise 12.5

- GCC 4.8.5
- WinCC OA Version 3.16 inklusive C++-Programmierschnittstelle
- CMake mit mindestens Version 3.5
- Den SMF_3P Ordner von der CD-ROM/DVD

Kapitel 2

Multi-Agent-SCADA-Systeme

Im folgenden Kapitel wird allgemein das Thema Multi-Agent-SCADA-Systeme beleuchtet und verschiedene Begriffe, wie sie in dieser Arbeit verstanden werden, definiert. Es werden die Anforderungen an solche Systeme zusammengefasst und Analogien zur Softwarelösung WinCC Open Architecture gezogen. Außerdem wird die Funktionsweise der WinCC OA Programmierschnittstelle anhand eines Beispiels erklärt.

2.1 SCADA - Supervisory Control And Data Acquisition

Unter SCADA kann man Softwarelösungen verstehen, die sich mit Datensenken in der *Feldebene* verbinden und somit einen Zugriff auf einzelnen Datenquellen und Senken, auch Eingabe- und Ausgabekanäle genannt, möglich machen. Das Schreiben und Lesen dieser Kanäle wird über eine eigene Echtzeitdatenbank ermöglicht. Eine Echtzeitdatenbank verarbeitet Transaktionen, garantiert einfache Korrektheitskriterien und arbeitet in Echtzeit unter Einhaltung von Zeitvorgaben [2]. Weitere Eigenschaften eines SCADA-Systems sind hier aufgelistet und zusammengefasst [8]:

- Kommunikation: Teilnehmer tauschen sich untereinander ereignisorientiert via TCP/IP aus (Publish/Subscribe [10]). Die Daten von Hardwaremodulen (Eingabe- und Ausgabekanäle) werden über Polling bezogen und in der Echtzeitdatenbank abgelegt.
- Skalierbarkeit: Hierbei versteht man die Möglichkeit, das System um weitere Instanzen mit jeweils eigenen Echtzeitdatenbanken zu erweitern.
- Redundanz: Um die Ausfallsicherheit zu erhöhen, muss ein Redundanzbetrieb möglich sein.
- Benutzerverwaltung: Verschiedene Benutzergruppen brauchen Zugriff auf den Datenhaushalt. Daher wird eine Benutzerverwaltung benötigt um die Sichtbarkeit bestimmter Daten einschränken zu können.
- Alarmbehandlung: Für ausgewählte Werte in der Datenbank können Hysteresen und Schwellwerte definiert werden. Sind diese über- oder unterschritten, werden Alarme ausgelöst und über verschiedene Kommunikationswege an Benutzerinnen oder Benutzer weiter gereicht. Tritt ein solcher Zustand ein, kann dies eine Benutzerinteraktion erfordern und mit einer Quittierungspflicht einhergehen.

- Archivieren: Damit ist die Mittel- und Langzeitspeicherung von Daten gemeint.
- Entwicklung: SCADA-Systeme zeichnen sich dadurch aus, dass sie Programmierschnittstellen zur Verfügung stellen, die Zugriff auf den Datenhaushalt (lesen und schreiben) ermöglichen. Außerdem bieten sie grafische Werkzeuge zum Modellieren der Echtzeitdatenbank.
- Automatisierung: Die ereignisorientierte Kommunikation in Verbindung mit der Programmierschnittstelle erlaubt das automatische Auslösen von Ereignissen.
- Objektorientierung: Damit ist das visuelle Anordnen des Datenhaushalts oder bereits das Gruppieren in der Echtzeitdatenbank gemeint. Eingabe- und Ausgabekanäle werden hierbei in logische Objekte, zum Beispiel Roboter, Turbine oder Ventil, gekapselt.
- Trends: Archivierte und Echtzeitdaten lassen sich über die Zeit grafisch visualisieren.

2.2 Agent

Obwohl es verschiedenste Literatur über Multi-Agent-Systeme gibt, wird der Begriff *Agent* nicht einheitlich definiert. In dieser Arbeit wird unter diesen Terminus ein Softwareprozess mit folgenden Eigenschaften verstanden [19]:

- Autonomie: Agenten agieren ohne direktes Eingreifen von Menschen oder anderen Komponenten und haben eine gewisse Kontrolle über ihre Handlungen und ihren internen Zustand.
- Soziale Fähigkeit: Sie interagieren mit anderen Agenten (und möglicherweise Menschen) über eine Art von Kommunikationsprotokoll.
- Reaktivität: Agenten nehmen ihre Umgebung wahr (das kann die physische Welt, eine Benutzerin oder ein Benutzer über eine grafische Benutzeroberfläche, eine Sammlung anderer Agenten, das Internet oder alles zusammen sein) und reagieren zeitnah auf Veränderungen.
- Proaktivität: Sie handeln nicht nur in Reaktion auf ihre Umwelt, sondern sind in der Lage, zielgerichtetes Verhalten an den Tag zu legen, indem sie die Initiative ergreifen.

Ein Multi-Agent-System kann daher als Sammlung von miteinander interagierenden Agenten interpretiert werden [17]. Demnach kann man als Multi-Agent-SCADA-System ein Multi-Agent-System, das die Eigenschaften eines SCADA-Systems besitzt, verstehen.

2.3 Simatic Windows Control Center Open Architecture

Während im letzten Abschnitt eine Definition für Multi-Agent-SCADA-System festgelegt wurde, wird in diesem nun die Umsetzung eines solchen anhand eines konkreten Softwareprodukts beschrieben. Das Basissystem WinCC Open Architecture stellt mit

Hilfe mehrerer Agenten eine Umgebung mit den vorher beschriebenen Funktionen zur Verfügung. Einen Überblick über das Zusammenspiel soll die Abbildung 2.1 geben:

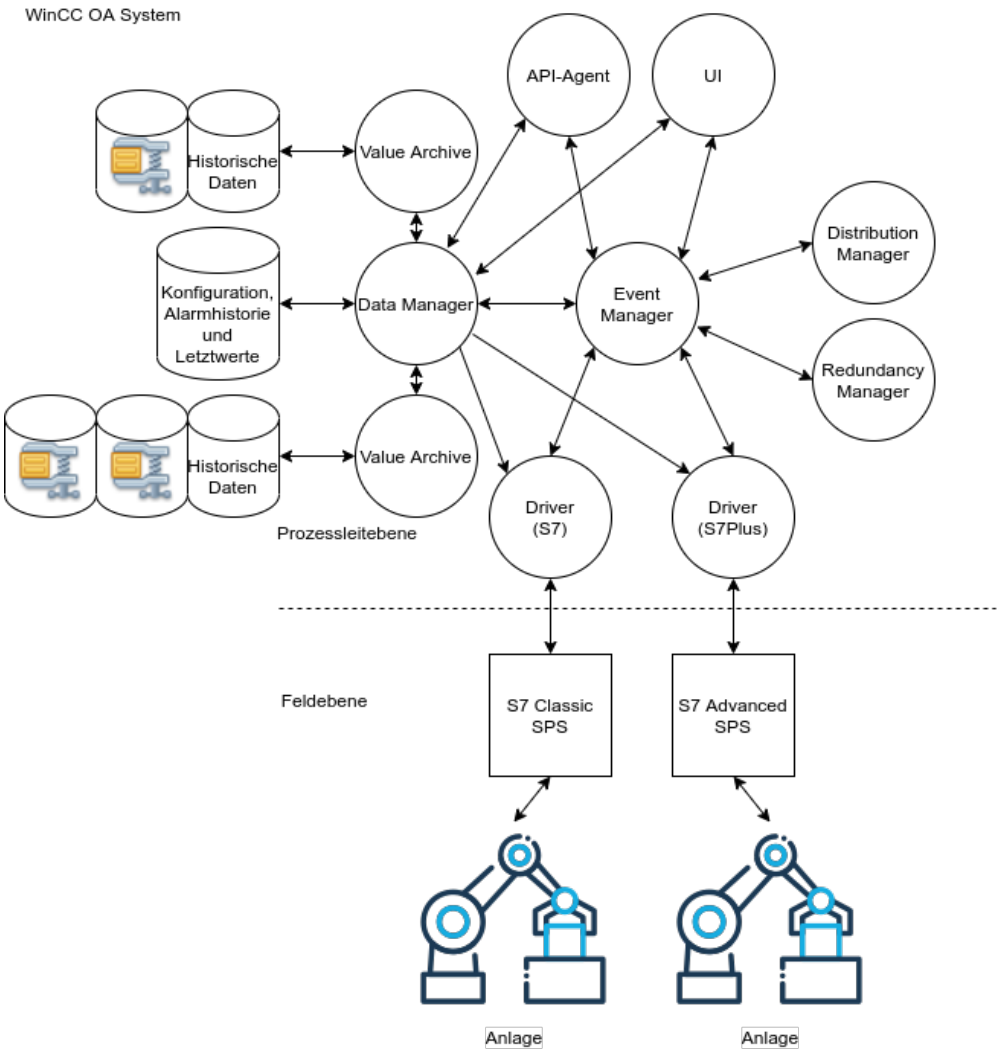


Abbildung 2.1: Ein WinCC OA System besteht aus verschiedenen funktionsspezifischen Agenten zusammen.

2.3.1 Basis Agenten und Konzept

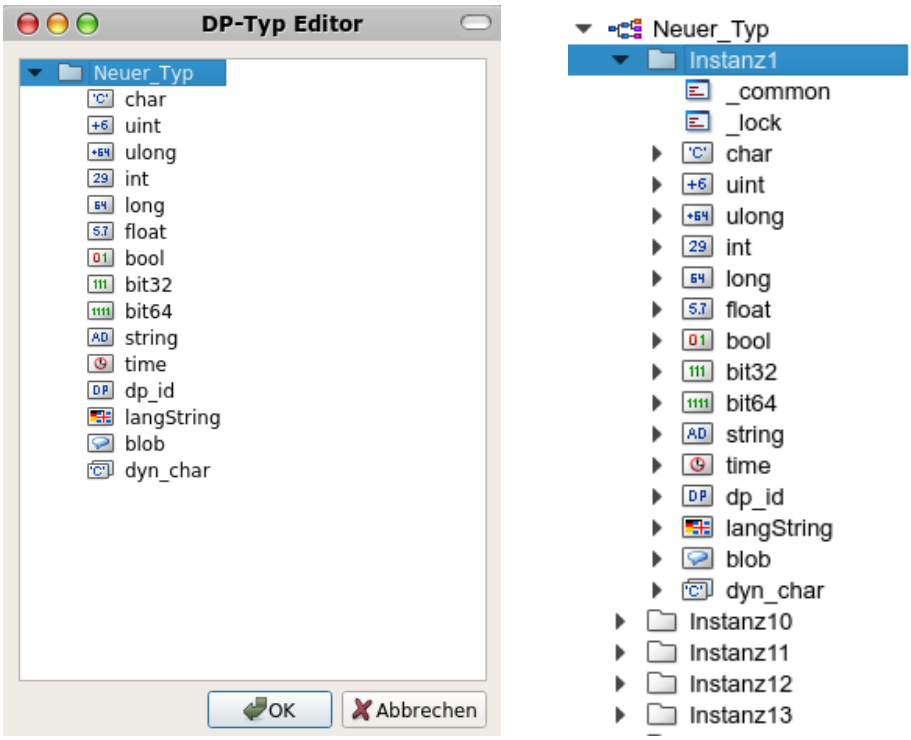
Je nach SCADA Anbieter werden die Teilnehmer eines solchen Systems unterschiedlich bezeichnet. Während in dieser Arbeit der Begriff *Agent* nach [19] Verwendung findet, werden diese in WinCC OA als *Manager* betitelt. Beide Ausdrücke folgen der vorher erläuterten Definition für *Agent*. Im kommenden Abschnitt werden die Basisprozesse eines WinCC OA Projekts beschrieben und somit die Grafik 2.1 genauer erklärt. Es wird dafür Bezug auf [5] genommen. Die einzelnen Teilnehmer kommunizieren untereinander über ein proprietäres TCP/IP-Protokoll.

Prozessmonitor

Der Prozessmonitor ist ein außenstehender Dienst und daher in der Systemübersicht 2.1 nicht repräsentiert. Er startet die einzelnen Prozesse eines WinCC OA Projektes und überwacht deren Status über zyklische Anfragen. Im Fehlerfall versucht er den fehlerhaften Prozess neu zu starten, um somit ein stabiles Gesamtsystem aufrecht zu erhalten.

UI Manager

WinCC OA besitzt mehrere Agenten dieses Typs. Sie ermöglichen die Konfiguration des Systems über grafische Eingabemasken. Einer der wichtigsten UI Manager wird *Para* bezeichnet und dient der Datenbankparametrierung.



(a) Datenpunkttyp-Editor

(b) Datenpunktinstanzen und deren Datenpunktelemente

Abbildung 2.2: Diese Abbildung zeigt die grafischen Bedienelemente zum Modellieren von Datenpunkttypen und Anlegen von Datenpunktinstanzen.

Die Daten werden in WinCC OA in sogenannte Datenpunkte gruppiert. Die Struktur eines Datenpunkts wird über Datenpunkttypen beschrieben. Diese können wie in Abbildung 2.2 gezeigt grafisch modelliert werden. Die Datenpunktelemente, die ein Datenpunkttyp gruppiert, besitzen statische Datentypen oder dynamische Listen dieser. Im Beispiel 2.2 sind alle von WinCC OA unterstützen Datentypen aufgezählt. In der Grafik 2.2 rechts sind mehrere Instanzen des links gezeigten Typs ersichtlich. Die einzelnen Unterknoten eines Datenpunktes werden als Datenpunktelemente bezeichnet. Diese

sind statische Datentypen und repräsentieren Werte innerhalb der Datenbank. Schreiben und Lesen erfolgt über folgende spezielle Attribute:

- _original: Soll ein Wert geschrieben werden, wird dieses Attribut verändert.
- _online: Erlaubt lesenden Zugriff auf den derzeitigen Prozesswert.
- _offline: Hiermit lassen sich historische Daten abfragen.

Mit Hilfe von *Para* ist es außerdem möglich, jedem Datenpunktelement zusätzliche Attribute hinzuzufügen. Dadurch ist eine erweitete Konfiguration möglich. Die wichtigsten sind:

- _alert_hdl: Erlaubt die Konfiguration von Hysteresen und Grenzwerten. Über- oder unterschreitet der Wert des Datenpunktelements diese Grenzen, löst der Event Manager einen Alarm aus.
- _address: Weist dieses Element einem Eingabe- oder Ausgabekanal in der *Feldebene* zu. Der entsprechende Driver-Prozess legt die Daten dann auf diesem ab.
- _archive: Erlaubt das Archivieren von Wertänderungen mit Hilfe eines Value-Archive-Prozesses. Hier wird hinterlegt, welcher Prozess für die Mittel- und Langzeitspeicherung zuständig ist.

Diese Konfigurationen werden an den Data-Manager-Prozess zum Ablegen in der Datenbank übertragen. Dieser gibt sie danach an alle Teilnehmer weiter, die sie benötigen. Über eine textuelle Repräsentation ist es dadurch möglich auf die Datenpunktelemente und deren Attribute zuzugreifen. Hier dazu ein Beispiel:

System1:Instanz1.int:_online..._value

Event Manager

Wie in Abbildung 2.1 ersichtlich ist der Event Manager die zentrale Kommunikationsinstanz eines WinCC OA Systems. Diese Einheit hält immer ein aktuelles Abbild der Prozessvariablen (Datenpunktelement) im Speicher. Jede andere Funktionseinheit, wie zum Beispiel eigene API-Agenten, erhalten auf Anfrage Daten aus dem Prozessabbild dieses Managers. Dadurch ist keine direkte Kommunikation mit den Teilnehmern in der *Feldebene* nötig. Es ist auch möglich sich ereignisorientiert auf Wertänderungen im Prozessabbild zu registrieren. Der Event Manager sendet hierbei die neuen Werte an alle registrierten Teilnehmer weiter. Da dieser Prozess die erste Instanz ist, die über Änderungen im Datenhaushalt verständigt wird, kümmert er sich auch um das Erzeugen von Alarmmeldungen. Ähnlich wie bei den Wertänderungen ist es Kommunikationspartnern möglich, sich für den Erhalt dieser Meldungen zu registrieren.

Data Manager

Der Data Manager unterstützt den Event Manager und kümmert sich um das persistieren des Prozessabbildes und der Alarmhistorie auf die Festplatte (Letztwertdatenbank). Außerdem speichert er die Informationen über Datenpunkttypen, Datenpunkte und die Konfigurationen der Datenpunktelemente. Diese gibt er an alle Teilnehmer weiter, die

diese benötigen. Anfragen über historische Daten übergibt der Event Manager an den Data Manager. Dieser reicht sie an die entsprechenden Value-Archive-Prozesse weiter und gibt die resultierenden Informationen an den Event Manager zurück.

Redundancy Manager

Um eine hohe Ausfallsicherheit garantieren zu können, ist ein WinCC OA Projekt redundant aufsetzbar. Die zwei Serversysteme kommunizieren miteinander mit Hilfe zweier Redundancy Manager.

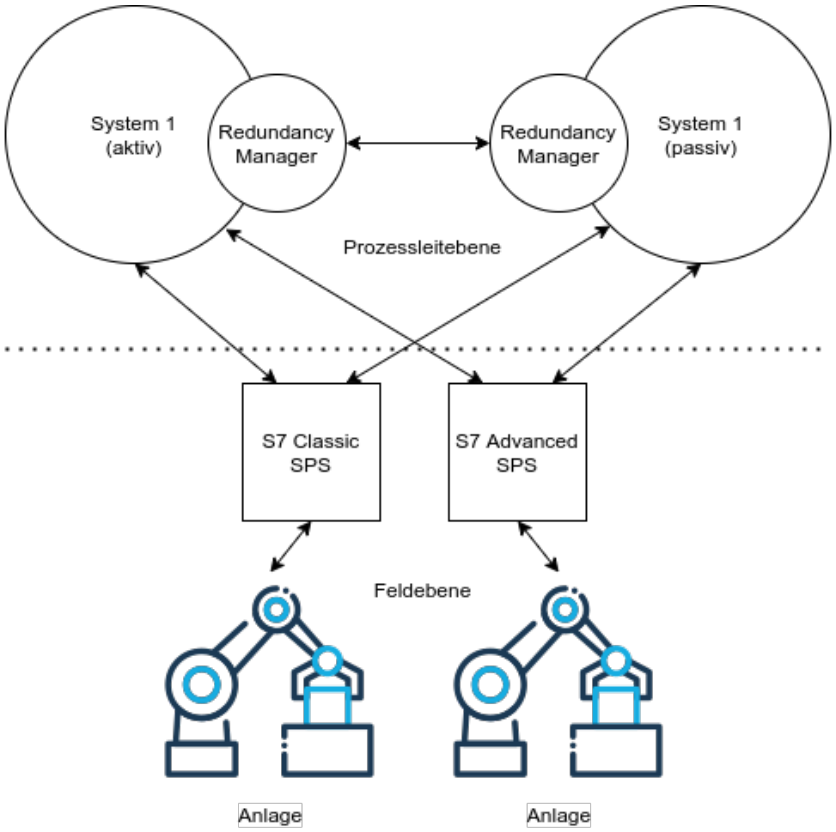


Abbildung 2.3: Ein redundant konfiguriertes System.

Wie in Abbildung 2.3 ersichtlich, besitzen beide Projekte dieselbe Konfiguration und sind mit derselben Peripherie in der *Feldebene* verbunden. Das passive System erhält alle Wertänderungen über die Redundancy Manager und verwirft die von den eigenen Teilnehmern. Bei einem Ausfall des aktiven Partners kann so automatisch der Passive einspringen.

Value Archive

Während der Data Manager die Funktion besitzt, die aktuellen Daten zu speichern, kümmern sich die Value Archives um die historische Aufzeichnung. Ist also für einzelne Datenpunktelemente eine Archivkonfiguration hinterlegt, so ist es die Aufgabe

des eingestellten Value Archives die Wertänderungen inklusive Zeitinformation in einer Datei abzulegen (Mittelzeitspeicherung). Erreichen die Archivdateien eine bestimmte Größe oder nach Verstreichen eines definierten Zeitraums, erfolgt ein Dateiwechsel und die alte Datei wird komprimiert. Komprimierte Archive kann man manuell auf entfernte Speichermedien exportieren (Langzeitspeicherung) und zu einem späteren Zeitpunkt für Analysezwecke importieren. Je nach Menge der zu archivierenden Daten können einem Projekt mehrere Value Archive Prozesse hinzugefügt werden.

Distribution Manager

Um größere Datenmengen verarbeiten zu können, ist es möglich, mehrere verteilte Projekte miteinander zu verknüpfen.

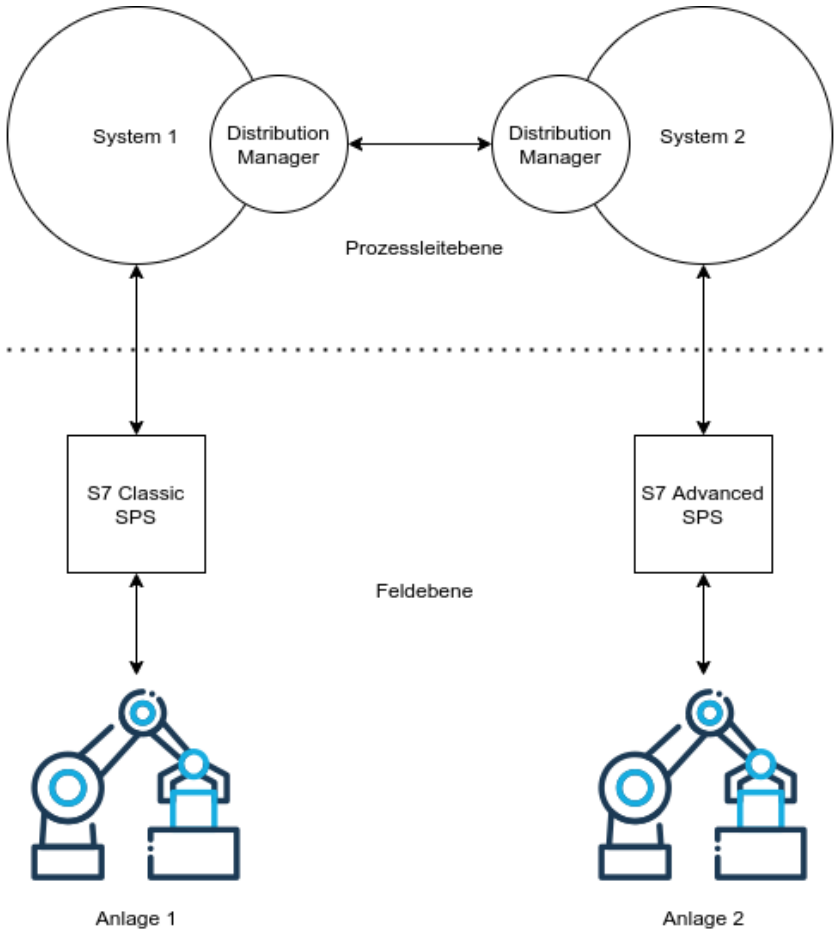


Abbildung 2.4: Zwei verteilte Systeme.

In der Grafik 2.4 ist erkennbar, dass die Verbindung der Systeme über zwei Distribution Manager realisiert wird. Somit ist es Agenten in System 1 möglich auf den Datenhaushalt von System 2 zuzugreifen und umgekehrt. Die Verbindung lässt sich aber auch unidirektional konfigurieren. Über diesen Mechanismus können bis zu 2048 Systeme miteinander verbunden werden.

Driver

Driver formen das unterste Level eines WinCC OA Systems. Sie sind für die Verbindung mit der *Feldebene* zuständig. Die verwendeten Peripheriegeräte oder die dazugehörigen Kommunikationsbusse bestimmen, welche Driver-Prozesse einem Projekt hinzugefügt werden müssen. Insgesamt werden 19 verschiedene Protokolle unterstützt, darunter MODBUS, S7Plus und OPC UA.

API-Agent

Diese Agenten können von Entwicklern über eine C++-Programmierschnittstelle erstellt werden. Man kann mit ihnen den Funktionsumfang des Systems erweitern. Oft werden Sie dazu verwendet, um ausgewählte Daten an die *Unternehmensleitebene* weiterzugeben oder um beim Einrichten eines Projektes automatisiert Datenbankkonfigurationen zu erstellen.

2.3.2 Projektaufbau und Hierarchie

Konfigurationsdateien

Eine der wichtigsten Dateien für ein WinCC OA Projekt ist die Konfigurationsdatei. Diese gliedert sich in mehrere Abschnitte, wo jeder Agent einen eigenen besitzt. Sie werden mit *[Agentenname]* eingeleitet, wobei Agentenname für den Namen der ausführbaren Datei, ohne Dateiendung steht. Dem folgen Schlüsselwertpaare, mit denen das Laufzeitverhalten der Agenten angepasst werden kann. Die Konfigurationsdatei wird einmalig beim Starten jedes Agenten geladen. Alle auf einem Rechner verfügbaren Projekte sind mit deren Pfaden in einer globalen Datei hinterlegt. Diese befindet sich auf Linux unter */etc/opt/pvss/pvssInst.conf* und auf Windows unter *C:/ProgramData/Siemens/WinCC_OA/pvssInst.conf*.

Subprojekte

Ein Hauptprojekt besteht aus mehreren Subprojekten. Diese sind alle innerhalb eigener Verzeichnisse organisiert.

Programm 2.1: Beispiel für die Auflistung der Subprojekte innerhalb einer Konfigurationsdatei.

```
1  [general]
2  pvss_path = "/opt/WinCC_OA/3.16" # WinCC OA Installationsverzeichnis
3  proj_path = "/home/giritzer/Beispiel/NGUI3.16" # niedrigste priorität
4  proj_path = "/home/giritzer/Beispiel/SMF_SmartScheduler"
5  proj_path = "/home/giritzer/Beispiel/SMF_PlantVisu"
6  proj_path = "/home/giritzer/Beispiel/ExampleProject" # höchste priorität
```

Die Ordnerstrukturen sind in der Projektkonfigurationsdatei innerhalb des speziellen Abschnittes *[general]* wie in dem Beispiel 2.1 ersichtlich hierarchisch angeordnet, wobei die höchste Ebene immer das Projektverzeichnis selbst darstellt. Existieren gleichnamige

Dateien in mehreren Hierarchieebenen, wird nur jene in der Höchsten berücksichtigt. Somit ist durch kopieren in das Hauptverzeichnis ein Anpassen möglich, ohne dabei die Originaldateien zu verändern. Über diesen Mechanismus können Kundenanforderungen und Anpassungen während einer Systemintegration umgesetzt werden.

2.3.3 C++-Programmierschnittstelle

WinCC OA stellt eine C++-Programmierschnittstelle zur Verfügung, mit der man eigene API-Agenten erstellen kann. Die Hauptfunktionen sind innerhalb zweier Singletonklassen gekapselt. Dieser Klassentyp hat die Eigenschaft, dass er innerhalb eines Programmes nur einmal instanziiert werden kann [13].

Resources-Klasse

Diese Klasse dient als Parser für die Programmparameter und die Projektkonfigurationsdatei. Die eingelesenen Parameter kann man über Zugriffsfunktionen (Getter) abfragen. Durch Ableiten von dieser Klasse ist ein Definieren von eigenen Konfigurationseinträgen für den zu entwickelnden Agenten möglich. Der Kommandozeilenparameter *-proj Projektname* weist diese Klasse an, über die globale *pvssInst.conf* Datei den Pfad zur Projektkonfigurationsdatei herauszufinden und diese zu laden. Mit *-num* wird dem Agenten eine eindeutige Nummer zugewiesen.

Manager-Klasse

Die *Manager-Klasse* beinhaltet alle Funktionen, um auf den Datenhaushalt des Systems zugreifen zu können. Um diese Klasse verwenden zu können, muss eine eigene Konkretisierung durch Ableiten implementiert werden. Folgendes Beispiel soll zeigen, wie dies zu bewerkstelligen ist:

Listing 2.1: Verwendung der Manager-Klasse.

```
0 class Agent : public Manager {
1 public:
2   Agent() :
3   // Rufe CTor der Elternklasse auf,
4   // Jeder Manager muss eindeutig
5   // über den ManagerIdentifier  identifizierbar  sein
6   Manager(ManagerIdentifier(
7           ManagerType::API_MAN,
8           Resources::getManNum())) {// verwendet geparsten '-num'
      Kommandozeilenparameter
9     // stelle  Verbindung zum System her
10    if(!connectToData(// Frage Datenbankkonfiguration an,
11                      // benötigt zum Übersetzten von
12                      // Datenpunktelementen -> DpIdentifier
13      StartDpInitSysMsg::TYPE_CONTAINER |
14      StartDpInitSysMsg::DP_IDENTIFICATION)) {
15      exit(EXIT_FAILURE);
16    }
17    while (getManagerState() == STATE_INIT) {
18      long sec = 1, usec = 0;
19      dispatch(sec, usec); // erhalte Daten
```

```
20     }
21     if(!connectToEvent()) {
22       exit(EXIT_FAILURE);
23     }
24   }
25 };
```

Der Programmcode 2.1 zeigt, dass jeder Teilnehmer des SCADA-Systems eine eindeutige Kennung besitzt. Diese nennt sich *ManagerIdentifier* und wird der Basisklasse über den Konstruktor mitgegeben. Wichtig ist die Nummer des Agenten, die innerhalb eines Systems eindeutig sein muss. In dem Beispiel 2.1 kann man erkennen, dass die *Resources*-Klasse Zugriff auf diese Nummer über die Funktion *Resources::getManNum()* ermöglicht. Im Konstruktor der *Agent*-Implementierung wird eine Verbindung mit dem Data Manager hergestellt, um die Datenbankkonfigurationen zu erhalten. Die Konfigurationen werden benötigt, um später die textuelle Repräsentation der Datenpunktelemente in interne Datentypen umwandeln zu können. Im letzten Schritt wird die Verbindung zum Event Manager hergestellt. Diese abgeleitete Klasse erbt alle Funktionen, um mit dem System interagieren zu können. Eine Auswahl ist in der folgenden Tabelle aufgelistet:

Tabelle 2.1: Wichtige Funktionen der Manager-Klasse

Funktion	*Beschreibung*
dpGet	Erlaubt das Abfragen der Werte eines Datenpunktelements oder mehrerer Datenpunktelemente.
dpSet	Ermöglicht das Ändern der Werte von Datenpunktelementen.
dpConnect	Kann für die Registrierung auf Wertänderungen verwendet werden.
dpGetPeriod	Funktion für eine historische Datenabfrage zwischen zwei Zeitpunkten.
alertGet	Liefert Informationen über aktive Alarme auf angegebenen Datenpunktelementen.
alertSet	Kann zum Auslösen oder Quittieren von Alarmen verwendet werden.
alertConnect	Erlaubt Registrierung zum Erhalt aller Alarme.
alertGetPeriod	Ermöglicht die Abfrage von historischen Alarmdaten.
dispatch	Führt asynchrone Kommunikation mit dem Gesamtsystem aus. Diese Funktion muss in einer Hauptschleife zyklisch aufgerufen werden.

Die in Tabelle 2.1 gezeigten Funktionen erhalten, bis auf *dispatch()*, die angefragten Daten asynchron. Die Antworten werden mit Hilfe des Observer-Entwurfsmuster übermittelt [4]. Jeder Funktionsaufruf benötigt ein Observer-Objekt, das die Daten erhält. Dazu muss das von der WinCC OA Programmierschnittstelle zur Verfügung gestellte *HotLinkWaitForAnswer*-Interface implementiert werden.

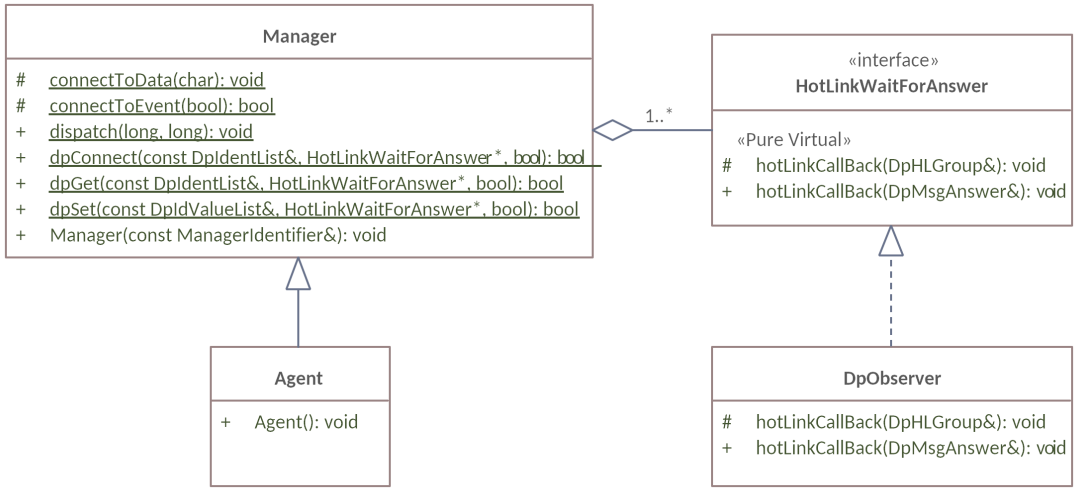


Abbildung 2.5: Zusammenspiel zwischen Manager-Klasse und *HotLinkWaitForAnswer*-Interface.

Die Abbildung 2.5 gibt einen Überblick, wie über das *Manager*-Singleton angefragte Informationen durch Ableiten der Interface-Klasse (DpObserver) asynchron erhalten werden können. Eine konkrete Implementierung ist im nachfolgenden Codebeispiel veranschaulicht:

Listing 2.2: Observer-Klasse, erhält asynchrone Antworten.

```
0  class DpObserver : public HotLinkWaitForAnswer {
1  public:
2    // Diese Funktion wird bei jeder Wertänderung eines
3    // Datenpunktelements (nach DpConnect) aufgerufen
4    void hotLinkCallBack(DpHLGroup &group) override {
5      std::cout << "--- hotLinkCallBack(DpHLGroup) ---" << std::endl;
6      TimeVar ts = group.getOriginTime(); // Zeitstempel der Antwort
7      // Ändert sich ein Datenpunktelement werden immer alle bei dem
8      // DpConnect registrierten mitübertragen, iteriere über alle Elemente.
9      for (DpVCItem* item = group.getFirstItem(); item;
10           item = group.getNextItem()) {
11       Variable* value = item->getValuePtr(); // Neuer Wert
12       CharString dpe = item->getDpIdentifier().toString(); // Name DPElement
13       // Erkenne Variablentyp und gebe Werte aus
14       std::cout << dpe << std::endl;
15       if(value->isA(VariableType::BIT_VAR) == VariableType::BIT_VAR){
16         std::cout << static_cast<BitVar*>(value)->getValue() << std::endl;
17       } else if (value->isA(VariableType::FLOAT_VAR) == VariableType::FLOAT_VAR) {
18         std::cout << static_cast<FloatVar*>(value)->getValue() << std::endl;
19       } else {
20         std::cout << "Unknown Variable type." << std::endl;
21       }
22     }
23   }
24   // Diese Funktion wird einmalig als Antwort einer
25   // einfachen Abfrage (DpConnect, DpGet, DpSet etc.) aufgerufen.
26   void hotLinkCallBack(DpMsgAnswer &answer) override {
```

```
27     std::cout << "--- hotLinkCallBack(DpMsgAnswer) ---" << std::endl;
28     for (AnswerGroup* grp = answer.getFirstGroup(); grp;
29           grp = answer.getNextGroup()) {
30       // Iteriere über Liste aller angefragten Datenpunktelementen
31       for (AnswerItem* item = grp->getFirstItem(); item;
32             item = grp->getNextItem()) {
33         TimeVar ts = item->getTime(); // Zeitstempel, als der Wert geschrieben wurde
34         Variable* value = item->getValuePtr(); // Derzeitiger Wert
35         CharString dpe = item->getDpIdentifier().toString(); // Name DPElement
36         // Erkenne Variablentyp und gebe Werte aus
37         std::cout << dpe << std::endl;
38         if(value->isA(VariableType::BIT_VAR) == VariableType::BIT_VAR){
39           std::cout << static_cast<BitVar*>(value)->getValue() << std::endl;
40         } else if (value->isA(VariableType::FLOAT_VAR) == VariableType::FLOAT_VAR) {
41           std::cout << static_cast<FloatVar*>(value)->getValue() << std::endl;
42         } else {
43           std::cout << "Unknown Variable type." << std::endl;
44         }
45       }
46     }
47   }
48 };
```

In dem Beispiel 2.1 ist gezeigt wie die Observer-Klasse zu implementieren ist. Wenn durch einen *dispatch()*-Aufruf Daten erhalten wurden, wird abhängig von der Abfrage die entsprechende *hotLinkCallback()*-Funktion verwendet. Wurde sich auf Wertänderungen registriert, wird bei jedem neuen Wert *hotLinkCallback(DpHLGroup)* aufgerufen. Für einmalige Antworten werden die Informationen via *hotLinkCallback(DpMsgAnswer)* übergeben. Die abgefragten Daten werden als Objekte der Klasse *Variable* übermittelt. Dies ist eine Basisklasse, von der für jeden verfügbaren statischen Datentyp der Datenbank eine Konkretisierung vorhanden ist. Um den tatsächlichen Wert zu erhalten, ist daher ein *Cast* auf den entsprechenden konkreten Typ nötig. Wie im Beispiel 2.1 ersichtlich, besitzt die *Variable*-Klasse für diesen Zweck die *isA()*-Funktion, mit dieser der richtige Datentyp ermittelt werden kann. Analog zum gezeigten Programmcode muss für die Alarmfunktionen das Interface *AlertHotlinkWaitForAnswer* implementiert werden.

Beispiel

Die zuvor gezeigten Klassen werden hier nun in einem Beispielprogramm verwendet, um die Funktionsweise zu veranschaulichen:

Listing 2.3: Hauptfunktion, zeigt die Funktionsweise der WinCC OA Programmierschnittstelle.

```
0 int main(int argc, char *argv[]) {
1   // Parst Konfigurationsdatei und Kommandozeilenparameter
2   Resources::init(argc, argv);
3   Agent* agt = new Agent();
4
5   // Erstelle eine Liste von Datenpunktelementen
6   // die verwendet werden sollen.
7   DpIdentList dpes; DpIdentifier dpe1, dpe2;
8   Agent::getId("System1:ExampleDP_DDE.f1:_online.._value", dpe1);
```

```
 9   Agent::getId("System1:ExampleDP_DDE.b1:_online.._value", dpe2);
10   dpes.append(dpe1); dpes.append(dpe2);
11
12   // Beispiel API Aufrufe, erhalte Werte
13   Agent::dpGet(dpes, new DpObserver(), true);
14   // registriere erhalt von Wertänderungen
15   Agent::dpConnect(dpes, new DpObserver(), true);
16
17   // setze Werte
18   DpIdValueList dpesAndValues;
19   Agent::getId("System1:ExampleDP_DDE.f1:_original.._value", dpe1);
20   Agent::getId("System1:ExampleDP_DDE.b1:_original.._value", dpe2);
21   dpesAndValues.appendItem(dpe1, new FloatVar(47.11));
22   dpesAndValues.appendItem(dpe2, new BitVar(false));
23   Agent::dpSet(dpesAndValues, new DpObserver(), true);
24
25   // Hauptschleife, kümmert sich um die asynchrone Kommunikation.
26   while (true) {
27       long sec = 1, usec = 0;
28       Agent::dispatch(sec, usec);
29   }
30   delete agt; agt = nullptr;
31   return EXIT_SUCCESS;
32 }
```

In dem Quellcode 2.3 ist ersichtlich, dass zunächst die Kommandozeilenparameter an die *Resources-Klasse* über die *Resources::init()*-Funktion übergeben wurde. Über *Agent::getId()* ist es möglich die textuelle Repräsentation der Datenpunktelemente in den *DpIdentifier* Datentyp umzuwandeln, der für die spätere Verwendung benötigt wird. Hier fällt auf, dass für abfragende Befehle das Attribut *_online.._value* angehängt wurde und für Schreibende *_original.._value*. Der letzte boolesche Parameter in den Aufrufen *Agent::dpGet()*, *Agent::dpSet()* und *Agent::dpConnect()* weist die Funktionen an, die Observer-Objekte zu löschen sobald diese nicht mehr gebraucht werden. Das Beispiel produziert folgende Ausgabe:

Listing 2.4: Ausgabe des Beispiels.

```
prototyp42:--- hotLinkCallBack(DpMsgAnswer) ---
prototyp42:"System1:ExampleDP_DDE.f1:_online.._value"
prototyp42:10
prototyp42:"System1:ExampleDP_DDE.b1:_online.._value"
prototyp42:1
prototyp42:--- hotLinkCallBack(DpMsgAnswer) ---
prototyp42:"System1:ExampleDP_DDE.f1:_online.._value"
prototyp42:10
prototyp42:"System1:ExampleDP_DDE.b1:_online.._value"
prototyp42:1
prototyp42:--- hotLinkCallBack(DpMsgAnswer) ---
prototyp42:--- hotLinkCallBack(DpHLGroup) ---
prototyp42:"System1:ExampleDP_DDE.f1:_online.._value"
prototyp42:47.11
prototyp42:"System1:ExampleDP_DDE.b1:_online.._value"
prototyp42:0
```

Wie in 2.4 erkennbar wird viermal eine *hotLinkCallback()*-Funktion aufgerufen. Die ersten beiden Ausgaben sind die Antworten auf *Agent::dpGet()* und *Agent::dpConnect()*.

Beim dritten Aufruf ist erkennbar, dass keine Werte übermittelt wurden. Dies ist die Antwort auf den *Agent::dpSet()* Befehl. Der Letzte ist die Benachrichtigung über eine Wertänderung des Datenpunktelements.

Das komplette Beispiel befindet sich als CMake [30] Projekt auf der CD-ROM/DVD unter *Thesis/prototype1* und kann mit folgenden Befehlen übersetzt werden. Vorraussetzung ist hierbei eine Installation der WinCC OA Programmierschnittstelle.

SUSE Linux Enterprise 12.5:

```
1 $ mkdir build
2 $ cd ./build
3 $ cmake .. -DWINCCOA_INSTALL=/opt/WinCC_OA/3.16
4 $ cmake --build .
```

Windows 10:

```
1 $ mkdir build
2 $ cd ./build
3 $ cmake .. -T v140 -A x64 -DWINCCOA_INSTALL=C:/Siemens/Automation/WinCC_OA/3.16
4 $ cmake --build .
```

2.4 SiPlant als Erweiterung

Um webbasierende Visualisierungen zu ermöglichen, wurden unter Zuhilfenahme der WinCC OA C++-Programmierschnittstelle drei Agenten entwickelt.
Die Grafik 2.6 zeigt die WinCC OA Erweiterung *SiPlant*. Die ausführbaren Dateien und der HTML5-Quellcode sind innerhalb eines eigenen Subprojekts organisiert. Die drei Agenten erfüllen folgenden Zweck:

- SIS_HTTP: Stellt den Webbrowsern die in HTML5 implementierte Visualisierung zur Verfügung.
- SIS_Event: Ist für die *SiPlant*-Webseite der Eintrittspunkt zu einer erweiterbaren Websocket-Schnittstelle. Andere Agenten können sich bei diesem Prozess anmelden um Befehle entgegenzunehmen.
- SIS_NGUI: Übernimmt Websocket-Befehle die sich um Benutzeranpassungen kümmern (verwalten der Benutzerprofile etc.).

SiPlant kann über den Subprojekt-Mechanismus sukzessive erweitert werden.
SIS_HTTP erlaubt den Webbrowsern zugriff auf die HTML5-Dateien aller Subprojekte. Spezifische Agenten können der Webseite über SIS_Event eigene Websocket-Befehle zur Verfügung stellen. Der CD-ROM/DVD liegen zwei *SiPlant*-Erweiterungen bei, die sich diesem Konzept Bedienen:

- SMF_PlantVisu: Ermöglicht Leistungs- und Fehleranalysen in einer Produktion.
- SMF_SmartScheduler: Implementiert einen Kalender, der das Planen von Schichten und Ereignissen erlaubt.

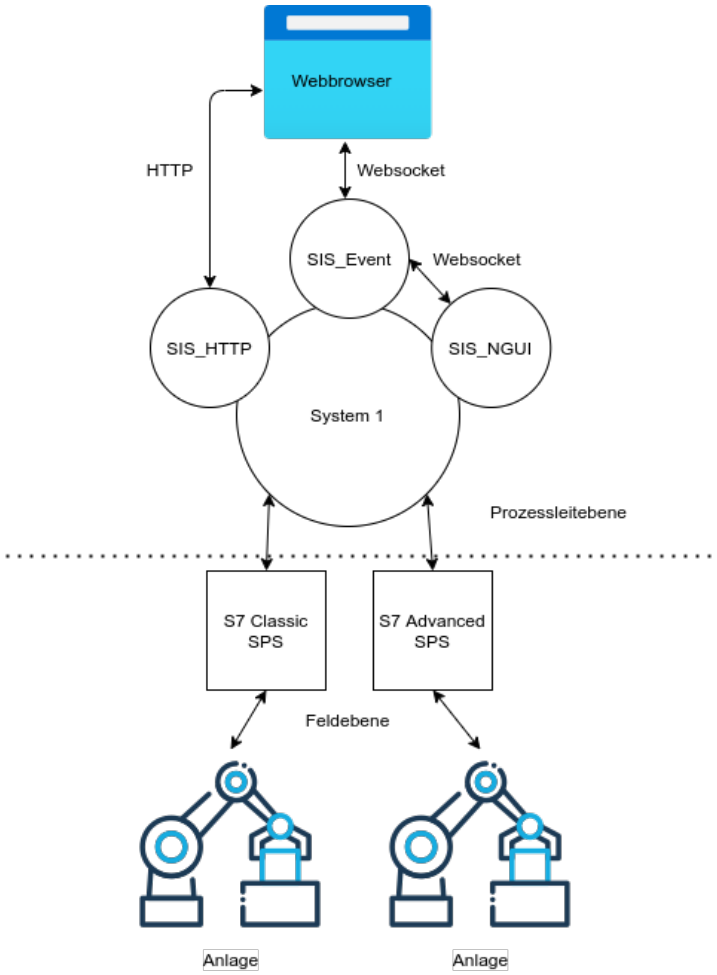


Abbildung 2.6: SIS_HTTP, SIS_Event und SIS_NGUI als Erweiterung für WinCC OA.

Kapitel 3

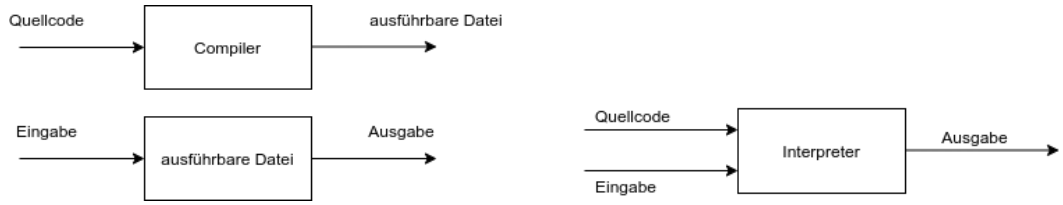
Google V8 Javascriptengine

Während im letzten Kapitel auf die Konzepte und Funktionsweise von SCADA-Systemen eingegangen wurde, widmet sich dieses nun der Skriptsprache Javascript. Es wird gezeigt, wie diese Sprache technisch funktioniert. Als Javascriptengine bezeichnet man Programmkomponenten, die ECMA standardkonformen Quellcode ausführen können [33]. Danach werden die drei bekanntesten Engines verglichen, um zu erläutern, warum die Google V8 Engine für die Umsetzung gewählt wurde. Im letzten Abschnitt werden die Konzepte, Funktionsweise und Programmierschnittstelle dieser beschrieben.

3.1 Grundlagen

Um zu verstehen, wie eine Javascriptengine funktioniert, müssen die Grundlagen der Compilertheorie umrissen werden. Dazu wird hier Bezug auf [1] und [18] genommen.

3.1.1 Sprachprozessoren



(a) Compiler **(b)** Interpreter

Abbildung 3.1: Diese Abbildung zeigt die Sprachprozessoren Compiler und Interpreter im Vergleich.

Als Compiler bezeichnet man einen Sprachprozessor, der Quellcode von einer Sprache in eine andere übersetzt. In der Regel ist die Zielsprache eine Sammlung von Instruktionen gebündelt in einer ausführbaren Datei. Diese kann wie in Abbildung 3.1 links gezeigt, gestartet werden, um Eingaben zu verarbeiten und Ausgaben zu generieren. Ein Interpreter ist ein anderer Typ von Sprachprozessor, der im Gegensatz zum Compiler keine Übersetzungen erzeugt, sondern den Quellcode direkt ausführt. Ein Beispiel

dafür ist in der Grafik 3.1 rechts illustriert. Für bestimmte Programmiersprachen wie Javascript sind auch Mischformen möglich. Beide Typen eint die Eigenschaft, dass sie im ersten Schritt das Quellprogramm analysieren müssen, um eine für sie verständliche Zwischensprache zu erzeugen. Diese Aufgabe übernimmt das *Frontend*.

3.1.2 Frontend

Als *Frontend* wird in der Compilertheorie jene Komponente bezeichnet, die eine Programmiersprache in eine maschinenunabhängige Zwischendarstellung übersetzt.

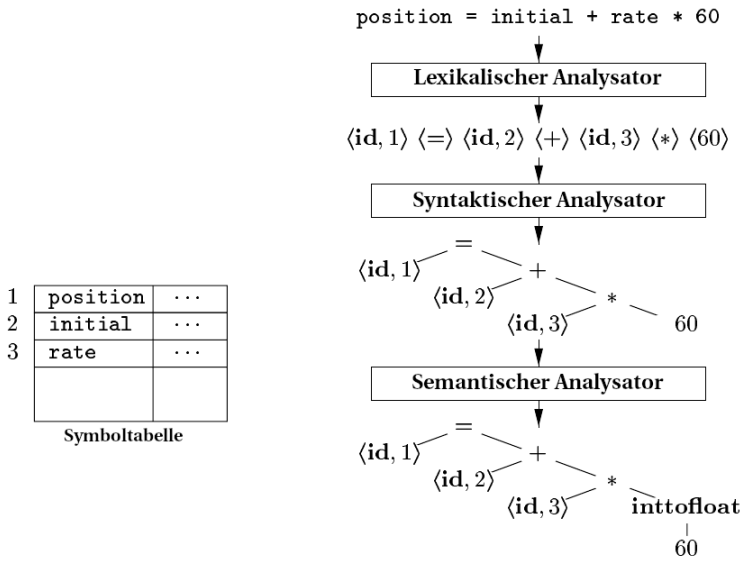


Abbildung 3.2: Schritte des Frontends [20].

Wie in der Grafik 3.2 ersichtlich, werden dazu drei Schritte durchgeführt. Ziel ist es, einen abstrakten Syntaxbaum zu erhalten, der für die spätere Codeerzeugung oder eine direkte Ausführung verwendet werden kann.

Lexikalische Analyse

Damit bezeichnet man das Einlesen eines Programmcodes, um die Zeichenketten in sinnvolle Sequenzen, sogenannte Lexeme, zu gruppieren und entsprechende Token zu erzeugen. Als Lexem wird eine einzelne sprachliche Bedeutungseinheit verstanden. Um Schlüsselworte, Operatoren und Symbole erkennen zu können, müssen deterministische Automaten implementiert werden, die Sprachmuster feststellen können. Auf Basis dieser Muster werden Token mit optionalen Attributen generiert, die an die syntaktische Analyse weitergegeben werden. Lexikalische Analysatoren werden auch *Lexer* oder *Scanner* genannt. Die Symbole einer Sprache werden in diesem Prozess in einer eigenen Tabelle abgelegt, die später in der semantischen Analyse Verwendung findet. Besitzt ein Token einen Attributwert, so verweist dieser auf ein Element in der sogenannten Symboltabelle.

Syntaktische Analyse

Die Syntaktik beschreibt Regeln, auch Grammatik genannt, nach denen Texte strukturiert werden. Aus diesem Grund wird in dieser Phase aus den Token des *Scanners*, wie in Abbildung 3.2 gezeigt, eine Baumstruktur erstellt, die den grammatikalischen Aufbau der Eingabe wiederspiegelt. Dies wird durch Parsen der Ausdrücke und Statements erreicht. Weicht das Quellprogramm von der Grammatik ab, so wird dies als Syntaxfehler erkannt. Diese Komponente wird auch *Parser* genannt.

Semantische Analyse

Unter Semantik wird die Zuordnung von Bedeutung zu Texten verstanden. Daher wird in dieser Phase mit Hilfe des Syntaxbaumes und der Symboltabelle die semantische Konsistenz des Programmes überprüft. Es werden Gültigkeitsbereiche verwaltet und die Einhaltung von Kontextbedingungen verifiziert. Ist dies nicht der Fall, so wird ein semantischer Fehler erzeugt. Falls die Sprachdefinition bestimmte Typkonvertierungen erlaubt, werden diese umgesetzt. Zum Beispiel kann ein binärer arithmetischer Operator entweder auf Ganzzahlen oder auf Gleitkommazahlen angewendet werden. Werden die Zahlentypen vermischt, kann der Compiler eine Umwandlung vornehmen. In Abbildung 3.2 ist dies unter der Annahme, dass *initial* und *rate* Gleitkommazahlen repräsentieren durch die eingefügte *inttofloat* Operation erkennbar.

3.1.3 Umsetzung in Javascriptengines

Die Javascriptengines Google V8, Mozilla Spidermonkey und Apple JavaScriptCore verwenden alle dasselbe Konzept [16], [44], [37]. Es wird zunächst über ein Frontend aus dem Javascript-Quellcode ein abstrakter Syntaxbaum erstellt. Dieser wird in eine Zwischensprache mit den Namen *Bytecode* übersetzt, die von einer virtuellen Maschine ausgeführt werden kann. Zur Laufzeit wird ermittelt, welche Codesequenzen oft durchlaufen werden. Dieser sogenannte *Hot-Code* wird unmittelbar (Just-In-Time) in echte Prozessorinstruktionen übersetzt, die dann direkt von dem Prozessor der Zielplattform ausgeführt werden können. Der generierte Maschinencode wird danach für eine spätere Ausführung in einem Cache abgelegt. Da Javascript eine dynamische Skriptsprache ohne Typsicherheit ist, kann es jedoch passieren, dass an eine optimierte Funktion ein Datentyp übergeben wird, der von dieser nicht verwendet werden kann. Tritt dieser Fall ein, muss auf den *Bytecode* zurückgegriffen werden.
Das Beispiel 3.1 zeigt, dass in Zeile 11 deoptimiert werden muss, da der übergebene Typ nicht dieselbe Struktur besitzt, wie in den vorhergehenden Aufrufen.

3.2 Wahl der Javascriptengine

Konzeptionell funktionieren die Javascriptengines Google V8, Mozilla Spidermonkey und Apple JavaScriptCore wie bereits erläutert gleich, daher sind sie auch ähnlich performant in der Ausführung von Javascript-Code. Ein Unterscheidungsmerkmal ist die Verbreitung weltweit, deshalb werden in diesem Abschnitt die Marktanteile aller vorher genannten Engines anhand der Verwendung in verschiedenen Webbrowsern verglichen.

Programm 3.1: Deoptimierung in Javascript.

```
1 // Testfunktion
2 function getX(obj) {
3   return obj.x;
4 }
5
6 // führe Funktion öfter aus,
7 // erzeugt Maschienencode
8 getX({ x: 4, y: 1 });
9 getX({ x: 7, y: 2 });
10 getX({ x: 1, y: 3 });
11 getX({ x: 1, y: 4 });
12
13 // hier muss deoptimiert werden
14 getX({ x: 1, z: 4 });
```

Tabelle 3.1: Marktanteile verschiedener Webbrowser weltweit [32], [28].

Browser	*Javascriptengine*	*netmarketshare.com*	*statcounter.com*
Google Chrome	V8	65,17%	65,73%
Apple Safari	JavaScriptCore	13,75%	18,04%
Mozilla Firefox	Spidermonkey	2,25%	3,54%
Microsoft Edge	V8	4,03%	3,5%
Samsung Internet	V8	3,07%	3,22%
Opera	V8	1,79%	2,17%

In der Tabelle 3.1 sind die prozentuellen Marktanteile der fünf meistgenutzten Webbrowser auf Mobil- und Desktopgeräten aufgelistet. Chrome, Edge, Samsung Internet und Opera verwenden alle die Google V8 Javascriptengine zum Ausführen von Javascriptdateien. Somit wird sie mit 74,06% laut netmarketshare.com und 75,17% laut statcounter.com in mehr als der Hälfte aller Webbrowser im Internet eingesetzt. Außerdem wird sie als Standardengine für das NodeJS Projekt verwendet [43]. Damit ist sie die weltweit am meisten verwendete Programmkomponente die sich um das Ausführen von Javascript-Code kümmert.

3.2.1 Fazit

Aufgrund der hohen Verbreitung in quelloffenen und kommerziellen Projekten kann davon ausgegangen werden, dass, obwohl die Google V8 Engine eine freie Lizenz besitzt, eine gute Wartung der Codebasis und ein langer Support garantiert ist. Aus diesem Grund wurde diese Softwarekomponente den anderen vorgezogen.

3.3 Google V8 Javascriptengine im Detail

Bevor das Umsetzungskonzept erläutert werden kann, muss die generelle Funktionsweise der Google V8 Javascriptengine erörtert werden. Dazu wird in diesem Abschnitt Bezug auf [41], [40], und [42] genommen. Gezeigt werden die einzelnen Ausführungsschritte anhand eines einfachen Codebeispiels, dass mit Hilfe der V8-Entwicklerkonsole *d8* ausgeführt wird. Diese kann mit dem Build-Skript in Anhang A gebaut werden. Dazu müssen jedoch die Parameter 'is_debug', 'is_clang' und 'use_sysroot' auf 'true' geändert werden. Das fertige Kompilat wird unter *v8/out/v8* abgelegt.

3.3.1 Abstrakter Syntaxbaum

Im ersten Ausführungsschritt muss zunächst, wie in dem Abschnitt 3.1 erklärt, mit Hilfe eines *Frontends* der Quellcode geparst und ein abstrakter Syntaxbaum erstellt werden. Jene Komponente die dies durchführt wird in der Google V8 Engine als *Parser* bezeichnet. Anhand der folgenden Funktion 'addiere' wird nun gezeigt, wie dieser Baum dargestellt wird:

Programm 3.2: Beispielcode, addiere.js

```
1 function addiere(a, b) {
2   return a + b;
3 }
4 addiere(1, 2);
```

Führt man das in 3.2 gezeigte Skript *addiere.js* in der V8-Entwickerkonsole mit dem Parameter *–print-ast* aus, so wird eine textuelle Repräsentation des abstrakten Syntaxbaums ausgegeben.

Programm 3.3: Textuelle Ausgabe der d8-Konsole

```
1 $ ./d8 --print-ast addiere.js
2 [generating interpreter code for user-defined function: addiere]
3 --- AST ---
4 FUNC at 16
5 . KIND 0
6 . SUSPEND COUNT 0
7 . NAME "addiere"
8 . INFERRED NAME ""
9 . PARAMS
10 . . VAR (0x55c826b31c40) (mode = VAR) "a"
11 . . VAR (0x55c826b31c70) (mode = VAR) "b"
12 . RETURN at 26
13 . . ADD at 35
14 . . . VAR PROXY parameter[0] (0x55c826b31c40) (mode = VAR) "a"
15 . . . VAR PROXY parameter[1] (0x55c826b31c70) (mode = VAR) "b"
```

In Abbildung 3.3 ist die Ausgabe 3.3 für ein besseres Verständnis grafisch visualisiert.

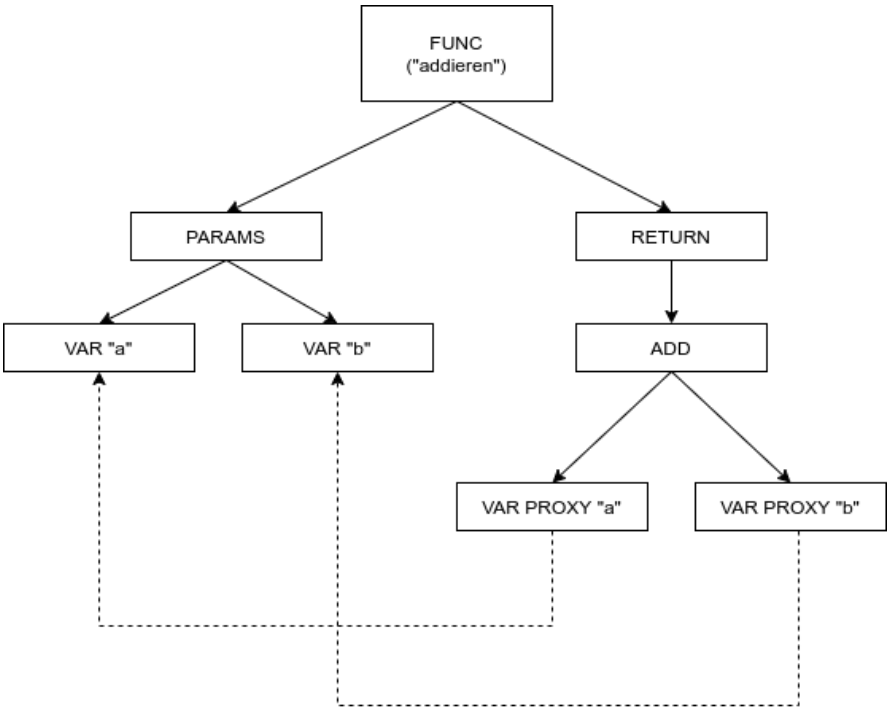


Abbildung 3.3: Abstrakter Syntaxbaum des Beispielcodes.

In der Visualisierung 3.3 kann man im linken Zweig die Funktionsparameter erkennen. Der Rechte beschreibt den Rückgabewert der Funktion. Hier fällt auf, dass die Kindknoten der arithmetischen Addition auf die Eingabeparameter referenzieren.

3.3.2 Interpreter

Aus diesem Baum wird im nächsten Schritt ein Bytecode generiert, der dann von dem Interpreter ausgeführt werden kann. Der Interpreter wird in der Google V8 Engine als *Ingnition* bezeichnet und ist als Registermaschine umgesetzt. Die meisten Bytecodes verwenden ein spezielles Register mit dem Namen *accumulator* um den zuletzt berechneten Wert zu speichern.
In der Ausgabe 3.4 ist erkennbar, dass zwei weitere Register, a1 und a0 verwendet werden. Diese entsprechen den Übergabeparametern auf dem Maschinenstapel. Die einzelnen Bytecodes in 3.4 können wie folgt interpretiert werden:

- StackCheck: Überpüft den Stapelzeiger auf Validität.
- Ldar a1: Lädt den Wert von Register a1 in das *accumulator* Register.
- Add a0, [0]: Lädt a0 und Addiert es auf den Wert im *accumulator* Register.
- Return: Gibt die Kontrolle zurück an den Aufrufer. Der Rückgabewert ist jener des *accumulator* Registers.

Programm 3.4: Bytecode der Funktion *addiere*.

```
1 $ ./d8 --print-bytecode addiere.js
2 [generating bytecode for function: addiere]
3 Parameter count 3
4 Frame size 0
5    16 E> 0x3a5e372c5ba @    0 : 91                StackCheck
6    26 S> 0x3a5e372c5bb @    1 : 1d 02             Ldar a1
7    35 E> 0x3a5e372c5bd @    3 : 2b 03 00          Add a0, [0]
8    39 S> 0x3a5e372c5c0 @    6 : 95                Return
9 Constant pool (size = 0)
10 Handler Table (size = 16)
```

3.3.3 Just-In-Time Compiler

Diese Komponente wird in der Google V8 Engine als *Turbofan* bezeichnet, da sie eine performantere Ausführung von Codeabschnitten ermöglicht. Wird vom Interpreter erkannt, dass eine Funktion oft aufgerufen wird, übergibt er den Bytecode an den Just-In-Time Compiler, damit dieser nativen Maschinencode erzeugt. Anschließend wird dieses Kompilat für eine spätere Ausführung im Speicher abgelegt. Dies kann mit der d8-Konsole über die *%OptimizeFunctionOnNextCall()* Laufzeitanweisung simuliert werden. Damit solche Anweisungen von der Entwicklerkonsole interpretiert werden, muss sie mit dem Parameter *–allow-natives-syntax* gestartete werden.

Programm 3.5: Optimierung und Deoptimierung.

```
1 function addiere(a, b) {
2   return a + b;
3 }
4
5 addiere(1, 2);
6 // Optimiere funktion und speichere
7 // sie im Cache
8 %OptimizeFunctionOnNextCall(addiere);
9 addiere(1, 2); // optimierter Aufruf
10 // 47.11 Falscher Typ -> deoptimiere
11 addiere(47.11, 2);
```

Der Javascript-Quellcode 3.5 zeigt, wie mit der Entwicklerkonsole ein Optimieren und Deoptimieren simuliert werden kann. Um diesen Prozess zu veranschaulichen, kann *d8* mit *–trace-deopt* gestartet werden.

In der Ausgabe 3.6 kann man in der Zeile drei erkennen, dass die optimierte Funktion ein Objekt des Typs *Smi*, also Small-Integer erwartet hätte. Da dieser aber im Beispiel 3.5 in der Zeile elf nicht übergeben wurde, löste dies eine Deoptimierung aus. Dabei werden der aktuelle Stackframe und die Hardwareregister ausgelesen, damit ein Frame für den Interpreter erstellt werden kann.

Programm 3.6: Ausgabe der Deoptimierung.

```
1 $ ./d8 --allow-natives-syntax --trace-deopt ./addiere.js
2   [deoptimizing (DEOPT eager): begin 0x3362fddac581 <JSFunction addiere (sfi = 0
      x3362fddac231)> (opt #0) @0, FP to SP delta: 16, caller sp: 0x7fff299d2698]
3   ;;; deoptimize at <./addiere.js:2:11>, not a Smi
4 reading input frame addiere => bytecode_offset=0, args=3, height=1; inputs:
5 0: 0x3362fddac581 ; [fp - 16] 0x3362fddac581 <JSFunction addiere (sfi = 0
      x3362fddac231)>
6 1: 0x35d8c4483231 ; [fp + 32] 0x35d8c4483231 <JSGlobal Object>
7 2: 0x3362fddac101 ; rax 0x3362fddac101 <Number 47.11>
8 3: 0x200000000 ; [fp + 16] 2
9 4: 0x3362fdd83cd1 ; [fp - 8] 0x3362fdd83cd1 <FixedArray[282]>
10 5: 0x27beca702569 ; (literal 1) 0x27beca702569 <Odd Oddball: optimized_out>
11 translating interpreted frame addiere => bytecode_offset=0, height=8
12 0x7fff299d2690: [top + 72] <- 0x35d8c4483231 ;  0x35d8c4483231 <JSGlobal Object>  (
      input #1)
13 0x7fff299d2688: [top + 64] <- 0x3362fddac101 ;  0x3362fddac101 <Number 47.11>  (
      input #2)
14 0x7fff299d2680: [top + 56] <- 0x200000000 ;  2  (input #3)
15 -------------------------
16 0x7fff299d2678: [top + 48] <- 0x1f5a07f1600 ;  caller's pc
17 0x7fff299d2670: [top + 40] <- 0x7fff299d26d8 ;  caller's fp
18 0x7fff299d2668: [top + 32] <- 0x3362fdd83cd1 ;  context    0x3362fdd83cd1 <
      FixedArray[282]>  (input #4)
19 0x7fff299d2660: [top + 24] <- 0x3362fddac581 ;  function    0x3362fddac581 <
      JSFunction addiere (sfi = 0x3362fddac231)>  (input #0)
20 0x7fff299d2658: [top + 16] <- 0x3362fddac659 ;  bytecode array 0x3362fddac659 <
      BytecodeArray[7]>  (input #0)
21 0x7fff299d2650: [top + 8] <- 0x3900000000 ;  bytecode offset @ 0
22 (input #0)
23 -------------------------
24 0x7fff299d2648: [top + 0] <- 0x27beca702569 ;  accumulator 0x27beca702569 <Odd
      Oddball: optimized_out>  (input #5)
25 [deoptimizing (eager): end 0x3362fddac581 <JSFunction addiere (sfi = 0x3362fddac231)
      > @0 => node=0, pc=0x1f5a07f1f00, caller sp=0x7fff299d2698, state=TOS_REGISTER,
      took 0.241 ms]
26 [removing optimized code for: 0x3362fddac581 <JSFunction addiere (sfi = 0
      x3362fddac231)>]
```

3.4 C++-Programmierschnittstelle

Nachdem nun bekannt ist, wie der Javascript-Code ausgeführt wird, widmet sich dieses Kapitel der C++-Programmierschnittstelle. Es werden die wichtigsten Konzepte erklärt und anhand eines Beispielprogrammes veranschaulicht. Dazu wird Bezug auf die Quellen [27], [48] und [38] genommen.

3.4.1 Isolates

Als *Isolate* bezeichnet man eine unabhängige Kopie der V8 Laufzeitumgebung, die einen eigenen Speicherbereich verwaltet. Ein Prozess kann mehrere dieser Objekte Instanziieren, aber jedes darf nur von jeweils einem Thread verwendet werden. Zu diesem

Zweck muss über Objekte des Datentyps *Locker* aus der V8-Programmierschnittstelle ein mehrfacher Zugriff verhindert werden. Diese funktionieren wie eine Mutal Exclusion und blockieren einen zweiten Thread, bis der erste den geschützten Bereich verlassen hat.

3.4.2 Contexts

Ein *Context* ist eine Ausführungsumgebung, die es ermöglicht, separaten, unabhängigen Javascript-Code in einer V8 Laufzeitumgebung (*Isolate*) auszuführen. Da ein *Isolate*-Objekt mehrere Context-Ausführungsumgebungen besitzen kann, muss jene, die zur Ausführung verwendet werden soll, explizit angegeben werden. Dies wird mit dem RAII-Konzept (Ressourcenbelegung ist Initialisierung) durch Erstellen eigener Objekte des Typs *Context::Scope* erreicht.

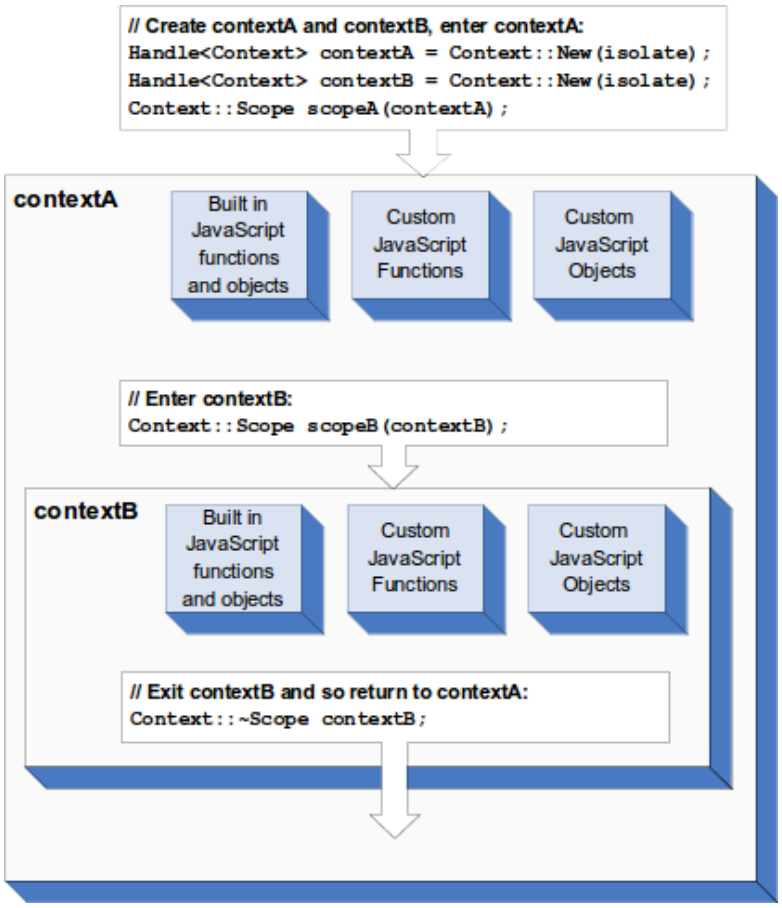


Abbildung 3.4: In dieser Illustration wird die Unabhängigkeit der verschiedenen Contexts zueinander veranschaulicht [22].

In Abbildung 3.4 ist ersichtlich, wie mehrere Ausführungsumgebung verschachtelt aufgerufen werden können ohne sich gegenseitig zu beeinflussen. Nach Aufrufen des Destruktors von dem scopeB Objekt, wird die Ausführung in contextA fortgesetzt.

3.4.3 Speicherverwaltung

Der Speicher innerhalb eines *Isolate* wird von einem Garbage-Collector verwaltet. Damit dieser weiß, welche Speicherbereiche noch in Verwendung sind, werden die Javascript-Objekte über sogenannte Handles überwacht. Diese halten eine Referenz auf die Position eines Javascript-Objekts in dem von einem *Isolate* verwalteten Speicherbereich. Gibt es kein Handle mehr, dass ein bestimmtes Objekt referenziert, kann dieses durch den Garbage-Collector freigegeben werden. Hier wird zwischen zwei Typen unterschieden:

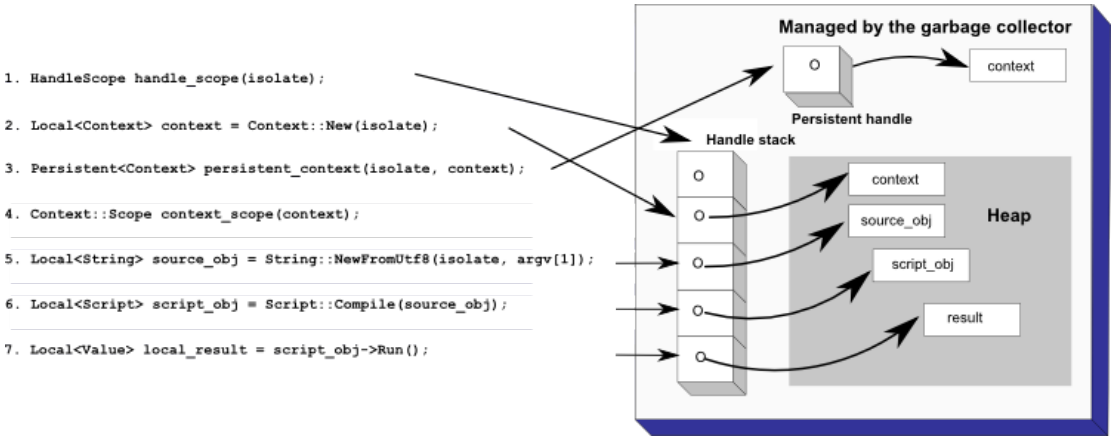


Abbildung 3.5: Speicherverwaltung mit Hilfe von Handles.[23]

Local

Ob ein Objekt noch valide ist, wird über das RAII-Konzept (Ressourcenbelegung ist Initialisierung) ermittelt. Dazu muss, bevor das Erzeugen von Javascript-Objekten möglich ist, wie in Abbildung 3.5 gezeigt, ein *HandleScope*-Objekt auf dem C++-Stack allokiert werden. Dieses Objekt stellt einen eigenen Stack für Local-Handles zur Verfügung. Wird es zerstört, zum Beispiel beim Verlassen des Gültigkeitsbereichs, werden alle Handles dieses *HandleScope*-Stacks gelöscht.

Persitent

Die Lebenszeit eines Persitent-Handles ist nicht an den aktuellen Gültigkeitsbereich gebunden. Daher muss man dem V8-Garbage-Collector manuell signalisieren, wenn es nicht mehr gebraucht wird. Dazu kann die *SetWeak()*-Funktion des Handle-Objekts aufgerufen werden. Dieser Mechanismus ist hilfreich, wenn ein Javascript-Objekt in mehreren C++-Funktionen verwendet werden soll.

3.4.4 Anpassen von Javascript-Ausführungsumgebungen über das Global-Objekt

Beim Erstellen eines neuen V8-Contexts kann ein Objekt des Datentyps *ObjectTemplate* angegeben werden, dass die Ausführungsumgebung des Contexts beschreibt. Diesem Objekt kann man beliebig viele eigene Funktionen (*FunctionTemplate*-Objekte) oder

Variablen zuweisen. Über *FunctionTemplate*-Objekte ist es wiederum möglich, C++-Funktionen aufzurufen. Dadurch lässt sich die Javascriptsprache beliebig erweitern. Auf Basis dieses *ObjectTemplate*-Objekts wird beim Erstellen der Ausführungsumgebung ein konkretes Objekt erstellt, das den globalen Javascript-Funktionsscope darstellt.

3.4.5 Das DebugInspector Protokoll

Die Google V8 Engine besitzt erweiterte Funktionen zur Fehlersuche über ein JSON basierendes Protokoll [31]. Die in den Webbrowsern Google Chrome und Microsoft Edge enthaltenen Entwicklerwerkzeuge unterstützen dieses und erlauben entfernten Zugriff auf V8-Instanzen via Websockets. Implementiert man die entsprechenden Interface-Klassen *V8InspectorClient* und *V8Inspector::Channel* der V8-Programmierschnittstelle, damit sie Befehle über einen Websocket-Server erhalten können, ist es möglich, den von der Engine ausgeführten Javascript-Code mit einen der zwei zuvor genannten Browser zu analysieren. Die *V8InspectorClient*-Implementierung muss die Verbindung zu einem *Context* herstellen. Danach können über den Websocket-Server erhaltene JSON-Befehle an die V8 Engine weitergegeben werden. Die *V8Inspector::Channel*-Umsetzung erhält die Antworten auf die Befehle und muss sie an den Befehlsgeber zurücksenden. Somit ist es möglich die Laufzeit eines laufenden Javascript-Codes zu analysieren und zu manipulieren. Außerdem kann man einzelne Funktionsaufrufe und Codesequenzen über die von den Browser-Entwicklerwerkzeugen zur Verfügung gestellte REPL (Read-eval-print loop) Konsole testen [3].

3.4.6 Beispiel

Nachfolgend wird anhand eines konkreten Beispiels gezeigt, wie die vorher erläuterten Konzepte angewendet werden, um innerhalb eines C++-Programmes Javascript-Quellcode ausführen zu können. Außerdem wird erklärt, wie die Skriptsprache über das *Global*-Objekt erweiterbar ist.

Listing 3.1: C++-Code, der über Javascript aufrufbar sein soll.

```
0 void PrintParams(const v8::FunctionCallbackInfo<v8::Value>& args) {
1   v8::Isolate* isolate = args.GetIsolate();
2   v8::Isolate::Scope iscope(isolate);
3   v8::HandleScope hscope(isolate);
4   // Überprüfe von Javascript übergebene parameter
5   if (args.Length() > 0) {
6     for (int i = 0; i < args.Length(); i++) {
7       // Konvertiere parameter in strings
8       // und gib diese aus
9       v8::String::Utf8Value str(args[i]);
10      std::cout << *str << std::endl;
11    }
12    args.GetReturnValue().Set(v8::Boolean::New(isolate, true));
13  }
14  else { // Werfe Javascript-Exception
15    isolate->ThrowException(v8::Exception::TypeError(v8::String::NewFromUtf8(isolate
      , "Invalid parameters")));
16  }
17 }
```

Die Funktion 3.1 soll Skripten zur Verfügung gestellt werden. Der C++-Quellcode nimmt alle von Javascript übergebenen Argumente entgegen und gibt sie über *std::cout* aus. In Zeile 12 wird an den aufrufenden Javascript-Code der boolesche Wert 'true' retourgegeben. Am Anfang des Beispiels ist erkennbar, dass ein HandleScope angelegt wird um alle Local Handles verwalten zu können. Hier wird keine Context::Scope Instanz benötigt, da das ausführende Skript bereits in einem aktiven Scope laufen muss. Wird kein Parameter übergeben, so wird eine Javascript-Exception geworfen. Diese kann im Skriptquellcode durch *try-catch*-Blöcke gefangen werden. Diese Funktion kann nun innerhalb eines Programmes an die V8 Engine übergeben werden:

Listing 3.2: Hauptfunktion, zeigt die Funktionsweise der V8-Programmierschnittstelle.

```
0 int main(int argc, char *argv[]) {
1   // Lade übergebene Datei
2   std::ifstream inFile(argv[argc-1]);
3   if (!inFile.good() || argc < 2) {
4     std::cerr << "Unable to open file " << argv[argc-1];
5     return EXIT_FAILURE;
6   }
7   std::stringstream source;
8   source << inFile.rdbuf();
9   std::string srcStr = source.str();
10
11  // Initialisiere V8 Engine
12  v8::V8::InitializePlatform(v8::platform::CreateDefaultPlatform());
13  v8::V8::Initialize();
14
15  // Erstelle ein neues Isolate, benutze den Standartallocator für Arrays
16  v8::Isolate::CreateParams create_params;
17  create_params.array_buffer_allocator = v8::ArrayBuffer::Allocator::
      NewDefaultAllocator();
18  v8::Isolate* isolate = v8::Isolate::New(create_params);
19
20  // Eigener Scope in dem das Isolate verwendet wird
21  {
22    v8::Locker locker(isolate); // Threadschutz/locking
23    v8::Isolate::Scope isolate_scope(isolate); // Scope gehört isolate
24    v8::HandleScope handle_scope(isolate); // Handles in diesem Scope gehören isolate
25
26    // Erstelle ein leeres global Objekttemplate für einen neuen Ausführungskontext
27    v8::Local<v8::ObjectTemplate> global = v8::ObjectTemplate::New(isolate);
28
29    // Füge eine funktion mit C/C++-Code zu diesem Objekttemplate hinzu
30    global->Set(v8::String::NewFromUtf8(isolate, "PrintParams", v8::NewStringType::
      kNormal).ToLocalChecked(), v8::FunctionTemplate::New(isolate, PrintParams));
31    // Erstelle einen Ausführungskontext, verwende das zuvor erstellte global Objekttemplate
      für das
32    // global Objekt.
33    v8::Local<v8::Context> context = v8::Context::New(isolate, NULL, global);
34
35    // Scope gehört context
36    v8::Context::Scope context_scope(context);
37
38    // Wandle Skript String in V8 Quellcode um
39    const v8::Local<v8::String> raw_script(v8::String::NewFromUtf8(isolate, srcStr.
      c_str()));
```

```
40      v8::ScriptCompiler::Source v8src(raw_script);
41
42      // Kompiliere Skript für context
43      v8::TryCatch tryCatchCompile(isolate); // tryCatch Objekt, fängt Exception
44      v8::Local<v8::Script> script = v8::ScriptCompiler::Compile(isolate, &v8src, v8::
        ScriptCompiler::kProduceCodeCache);
45
46      if (tryCatchCompile.HasCaught()) { // Gebe Compilefehler aus
47        ReportException(isolate, &tryCatchCompile);
48        return EXIT_FAILURE;
49      }
50      else { // Führe Skript aus
51        v8::TryCatch tryCatchRun(isolate);
52        v8::Local<v8::Value> result;
53        bool ret = script->Run(context).ToLocal(&result);
54        if (tryCatchRun.HasCaught() || !ret){ // Gebe Laufzeitfehler aus
55          ReportException(isolate, &tryCatchRun);
56          return EXIT_FAILURE;
57        }
58      }
59
60    } // Verlasse Scope -> isolate ist nun nicht mehr in Verwendung
61
62    // Gebe Resourcen frei
63    isolate->Dispose();
64    v8::V8::Dispose();
65    v8::V8::ShutdownPlatform();
66
67    return EXIT_SUCCESS;
68  }
```

Das Programm 3.2 lädt eine übergebene Javascriptdatei, kompiliert diese und führt sie aus. In Zeile 30 ist ersichtlich, wie einem *ObjectTemplate*-Objekt die Funktion *PrintParams* 3.1 hinzugefügt wird. Dieses Objekt wird danach in Zeile 33 zum Erstellen des Ausführungskontexts verwendet. Damit ist es von Javascript aus möglich, die C++-Funktion aufzurufen. Zum Überprüfen auf Kompilier- und Laufzeitfehler werden eigene *TryCatch*-Objekte verwendet. Tritt ein Problem auf, so wird die Fehlerinformation innerhalb dieser Objekte gesammelt. Die *ReportException*-Funktion gibt diese Fehler aus. Sie ist hier aus Platzgründen nicht aufgeführt, kann aber über Anhang C eingesehen werden.

Tests

Zur Veranschaulichung der Funktionalität dieses Beispiels wurden zwei kurze Programme in Javascript realisiert:

Listing 3.3: Skript test.js zum Testen der Funktion *PrintParams()*

```
1 var text = "Hello, World!";
2 var bool = false;
3
4 bool = PrintParams(text, bool);
5 PrintParams(text, bool);
6
7 for(var i = 0; i < 2; i++)
```

```
 8     PrintParams(text, bool, i);
 9
10 // wirft exception
11 PrintParams();
```

Dieses Programm gibt mehrmals verschiedene Werte und Variablen über die entwickelte *PrintParams()*-Funktion 3.1 aus. Zum Schluss wird diese Funktion ohne Parameter aufgerufen. Dadurch wird eine Javascript-Exception erzeugt, die von dem *tryCatchRun*-Objekt in der Hauptfunktion des C++-Quellcodes 3.2 gefangen wird.

Listing 3.4: Ausgabe des ersten Tests.

```
$ prototyp2 ./test.js
Hello, World!
false
Hello, World!
true
Hello, World!
true
0
Hello, World!
true
1
undefined:11
PrintParams();
TypeError: Invalid parameters
    at <anonymous>:11:1
```

Die Ausführung 3.4 zeigt ein korrektes Verhalten des Programmcodes. Es wurden alle Werte und der Exception-Text ausgegeben. Folgendes Skript soll das Verhalten bei einem Kompilierfehler zeigen:

Listing 3.5: Skript test2.js provoziert Kompilierfehler.

```
1 // Compile fehler
2 i =;
```

In 3.5 ist ein syntaktischer Fehler erkennbar. Das Starten dieses Programmes produziert folgende Ausgabe:

Listing 3.6: Ausgabe des zweiten Tests.

```
$ prototyp2 ./test2.js
undefined:2
i =;
SyntaxError: Unexpected token ;
```

Man kann erkennen, dass die Google V8 Enginen dieses Problem bereits zur Kompilierzeit identifizieren konnte und Hinweise zum Beheben ausgibt. Abgefangen wurde der Fehler über das *tryCatchCompile*-Objekt in der Hauptfunktion.

Das komplette Beispiel befindet sich als CMake [30] Projekt auf der CD-ROM/DVD unter *Thesis/prototype2* und kann mit folgenden Befehlen übersetzt werden. Zum Übersetzten wird der *SMF_3P* Ordner benötigt, da dieser die Bibliotheken der Google V8 Engine in Binärform beinhaltet. Alternativ können diese auch mit den Skripten in Anhang A erzeugt werden. Dieser Ordner befindet sich auch auf der CD-ROM/DVD.

SUSE Linux Enterprise 12.5:

```
$ mkdir build
$ cd ./build
$ cmake .. -DSMF_3P=~/SMF_3P
$ cmake --build .
```

Windows 10:

```
$ mkdir build
$ cd ./build
$ cmake .. -T v140 -A x64 -DSMF_3P=C:/SMF_3P
$ cmake --build .
```

Kapitel 4

Anforderungen

Der zu entwickelnde Agent soll es Systemintegratorinnen und Systemintegratoren ermöglichen, über Javascript auf ein WinCC OA SCADA-System zugreifen zu können. Dazu müssen die in den letzten Kapiteln beschriebenen Konzepte der Google V8 Engine und von WinCC OA verwendet werden. Im Folgenden sind die Anforderungen an das Programm definiert.

4.1 Zu unterstützende Plattformen

Einen limitierenden Faktor stellt hier die WinCC OA 3.16 Programmierschnittstelle dar, weil sie nur als Kompilat in Binärform zur Verfügung steht. Die unterstützten Plattformen sind:

- SUSE Linux Enterprise 12.5 (64 Bit)
- Astra Linux 1.6 (64 Bit)
- CentOS 7.9 (64 Bit)
- Red Hat Enterprise Linux 7.9 (64 Bit)
- Windows 10 (64 Bit)
- Windows Server 2019 (64 Bit)
- Windows Server 2016 (64 Bit)
- Windows Server 2012 (64 Bit)

Da CentOS ein gemeinnütziges Open-Source-Projekt ohne offiziellen Support ist [29] wird dessen Unterstützung nicht in Betracht gezogen. *SiPlant* ist auf Astra Linux und Red Hat Enterprise Linux aufgrund von Inkompatibilitäten einiger Systembibliotheken nicht lauffähig. Daher wird die Auswahl auf SUSE Linux Enterprise 12.5 und die von WinCC OA unterstützten Windows-Versionen eingeschränkt.

4.2 Anwendungsfälle

Das Programm soll die Realisierung von Kundenwünschen und die Anlagenintegration unterstützen. Dazu werden im Folgenden die genauen Anforderungen an den Agenten beschrieben.

4.2.1 Entwicklung kundenspezifischer Anforderungen

Oft stellen Kundinnen und Kunden während der Projektphase Anforderungen, die nicht im Standardfunktionsumfang von *SiPlant* enthalten sind. Um auf diese Wünsche agil eingehen zu können, ohne die Produktentwicklung zu stören, ist eine Möglichkeit der Umsetzung vor Ort nötig. Hier soll der V8-Agent eine Hilfe bieten, um Algorithmen implementieren zu können.

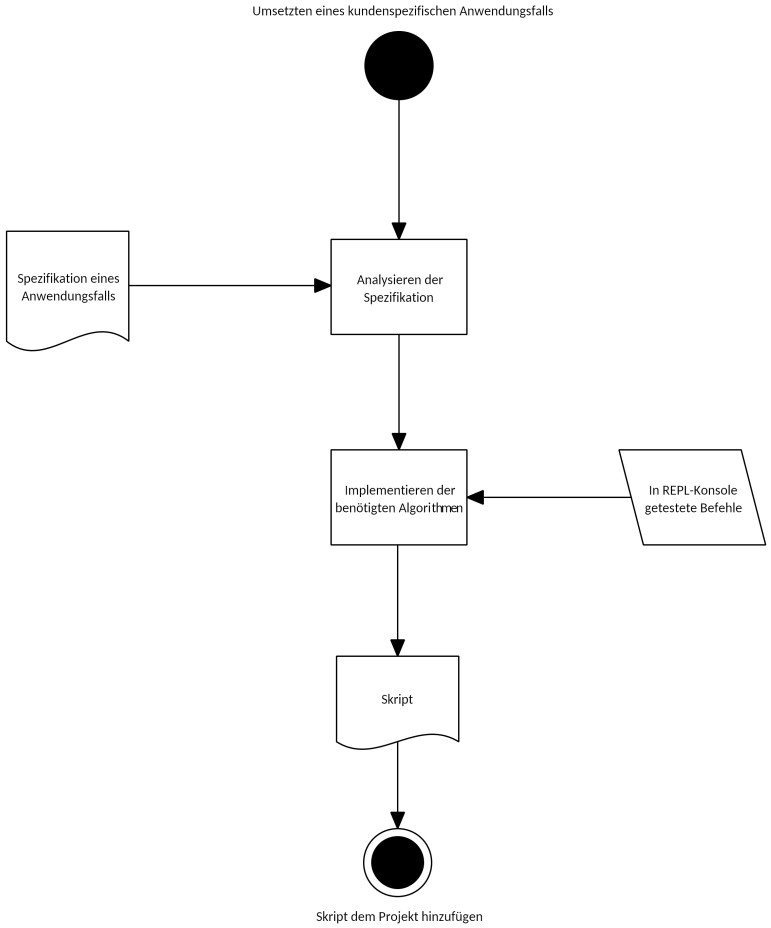


Abbildung 4.1: Dieses Ablaufdiagramm stellt die Entwicklung kundenspezifischer Anforderungen grafisch dar.

Dazu müssen wie in Abbildung 4.1 gezeigt, Kundinnen und Kunden zunächst eine Anforderung spezifizieren. Im nächsten Schritt ist es die Aufgabe der Systemintegratorinnen und Systemintegratoren diese Spezifikation zu analysieren. Danach können die entsprechenden Algorithmen unter Zuhilfenahme der REPL-Konsole implementiert werden. Diese müssen dem Projekt hinzugefügt werden. An dieser Stelle bietet sich der in 2.3.2 beschriebene WinCC OA Subprojekt-Mechanismus an, um eine gute Organisation und Wiederverwendbarkeit des erstellten Codes zu erreichen.

4.2.2 Skriptgestützte Massenparametrierung

Einen erheblichen Aufwand bei der Systemintegration nimmt das Anbinden der Teilnehmer in der *Feldebene* in Anspruch, weil hier wie in 2.3.1 beschrieben, für jeden Eingabe- und Ausgabekanal eine eigene Konfiguration vorgenommen werden muss. Der Agent soll durch eine skriptgestützte Parametrierung Erleichterung schaffen. Dazu wird ein Export von sogenannten Variablen- oder Taglisten der SPS-Programme benötigt.

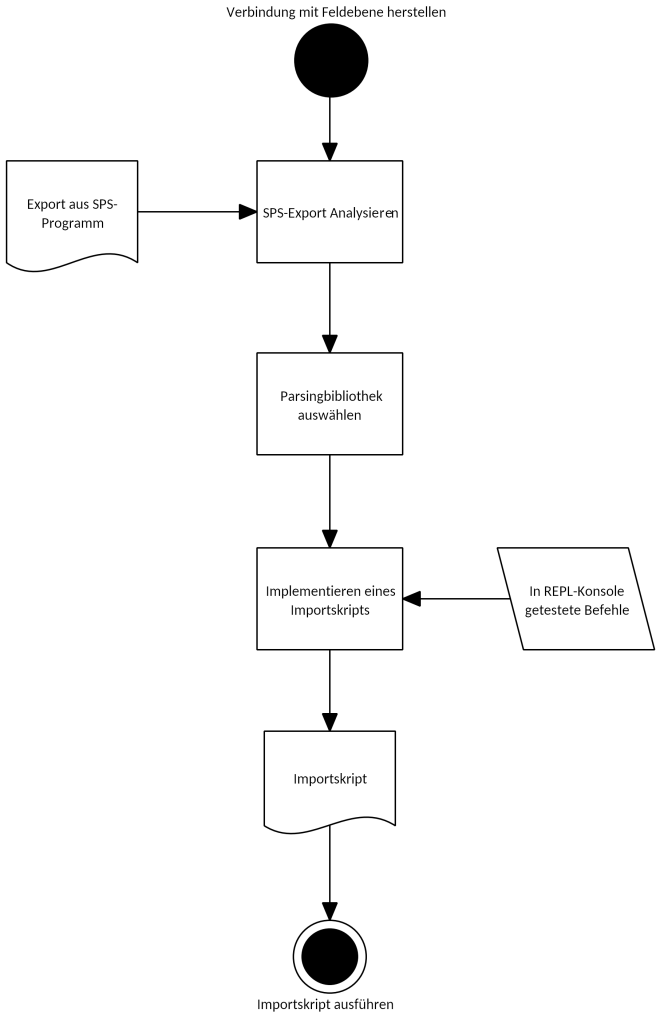


Abbildung 4.2: In diesem Diagramm wird die Massenparametrierung dargestellt.

Diese müssen wie in Abbildung 4.2 visualisiert von den SPS-Entwicklerinnen oder SPS-Entwicklern zur Verfügung gestellt werden. Häufig liegen diese Listen im CSV oder XML Format ohne standardisierte Struktur vor. Daher ist es die Aufgabe der Systemintegratorinnen und Systemintegratoren die Syntax dieser Dateien zu analysieren. Danach muss im Internet eine passend Javascript Parserbibliothek gefunden werden. Der nächste Schritt im Ablaufdiagramm sieht dann die Implementierung eines Import-Skripts vor.

Unterstützend kann dazu die REPL-Konsole der Browser-Entwicklerwerkzeuge herangezogen werden. Zuletzt wird der Import durch das Ausführen des Skriptes gestartet und die entsprechenden Datenpunkte und Datenpunktkonfigurationen erzeugt.

4.3 Qualitätsanforderungen

Hier werden alle Punkte definiert, die die Qualität des Programmes aus Sicht der Systemintegratorinnen und Systemintegratoren betreffen. Die Qualitätsanforderungen lassen sich in Benutzbarkeit, Wartbarkeit und Zuverlässigkeit unterteilen.

4.3.1 Benutzbarkeit

Digitales Handbuch für Systemintegratorinnen und Systemintegratoren als Nachschlagewerk

Um eine gute Benutzbarkeit garantieren zu können, muss den Systemintegratorinnen und Systemintegratoren eine digitale Dokumentation mit folgendem Inhalt vorliegen:

- Anleitung zum Verbinden der Google Chrome und Microsoft Edge Browser-Entwicklerkonsolen für Analysezwecke und zum Testen von Codesequenzen in der REPL-Konsole.
- Einfach verständliche Codebeispiele für jede durch den Agenten zur Verfügung gestellte Javascript-Funktion.
- Beschreibung aller Funktionen und Funktionsparameter.

4.3.2 Wartbarkeit

Der Agent muss erweiterbar sein, ohne bestehende Komponenten anpassen zu müssen. Um die Einarbeitungszeit für andere Entwicklerinnen und Entwickler zu verkürzen sind Kommentarblöcke im Quellcode nötig. Außerdem muss eine technische Dokumentation zur Verfügung stehen. Dies ist wichtig, damit neue, von Systemintegratorinnen oder Systemintegratoren angefragte Funktionen ohne großen Zeitaufwand ergänzt werden können.

4.3.3 Zuverlässigkeit

Die Benutzenden sollen sich darauf verlassen können, dass das Programm zuverlässig läuft und eventuell auftretende Bedienungsfehler seitens der Verwendenden zu keinem Absturz führen. Dazu müssen alle Eingaben, bevor eine Ausführung getätigt wird, auf deren Richtigkeit geprüft werden. Die Anwendung informiert die Benutzerin oder den Benutzer bei einem technischen oder einem Anwenderfehler. Die Fehlermeldungen müssen leicht verständlich sein und den Systemintegratorinnen und Systemintegratoren hinweise für die Richtigstellung geben. Als Beispiel kann hier ein Aufrufen von Funktionen mit falschen Parametern aufgeführt werden. Tritt eine solche Situation ein, muss durch eine Meldung Auskunft über die erwarteten Funktionsparameter gegeben werden. Ein weiterer wichtiger Punkt in der Benutzbarkeit ist die API-Stabilität. Unter diesem Begriff wird eine Garantie, dass sich bestehende Funktionssignaturen zwischen verschiedenen *SiPlant*-Versionen nicht verändern, verstanden.

4.4 Einsatz in spezifischeren C++-API-Agenten

Die Programmkomponente muss so konzipiert sein, dass sie auch in anderen SCADA-Agenten eingebunden und verwendet werden kann. Somit soll die Möglichkeit geschaffen werden, bestimmte Algorithmen in Skripte auslagern zu können, damit eine höhere Anpassbarkeit vor Ort in Projekten zustande gebracht wird. Als Beispiel können hier Priorisierungsalgorithmen für eingehende Aufträge aus der *Produktionsleitebene* genannt werden, da diese stark kundenabhängig sind.

Kapitel 5

Konzept

Im Folgenden wird ein Realisierungskonzept für die im letzten Kapitel beschriebenen Anforderungen erarbeitet. Es werden verschiedenen Strategien verglichen und ein Fazit gezogen. Im letzten Abschnitt wird die Machbarkeit anhand eines funktionalen Prototypen evaluiert.

5.1 Compiler

Wichtig ist hier zu erwähnen, dass vonseiten des WinCC OA SCADA Produktes für die jeweilige Plattform bestimmte Compiler vorgeschrieben sind, weil ansonsten aufgrund von etwaigen *ABI (Application Binary Interface)* Inkompatibilitäten mit der Programmierschnittstelle keine korrekte Funktionsweise garantiert wird. Für Linux ist in der WinCC OA Dokumentation ein C++-Compiler der *GNU Compiler Collection (GCC)* in der Version 4.8.5 vorgeschrieben. Zum Erzeugen von Windows Binärdateien wird der Compiler des *Visual Studio Toolset* in der Version v140 angegeben. Alle Programmkomponenten und Drittbibliotheken müssen mit diesen Compilern gebaut werden.

5.2 Linken der Google V8 Engine

Im folgenden Abschnitt wird erklärt, wie ein Programm mit externen Bibliotheken umgehen kann. Die verschiedenen Möglichkeiten werden abgewogen und in einem Fazit wird entschieden, welche Methode zum Linken der Google V8 Engine verwendet wird.

5.2.1 Statisches Linken

Wird eine Bibliothek statisch gelinkt, dann werden die Funktionen dieser Bibliothek direkt in die ausführbare Datei kopiert. Voraussetzung dafür ist, dass die gelinkte Bibliothek selbst statisch ist. Dies bedeutet, dass mindestens eine Objektdatei mit Hilfe des Kommandozeilenprogrammes *ar (archiver utility)* in ein Archiv umgewandelt wurde. *Ar* fügt zu einem solchen Archiv eine Tabelle hinzu, in der vermerkt ist welche Symbole, wo im Archiv zu finden sind [11].

5.2.2 Dynamisches Linken

Beim dynamischen Linken werden die Funktionen einer dynamischen Bibliothek in der ausführbaren Datei nur referenziert. Die dynamische Bibliothek wird erst dann geladen, wenn das Programm gestartet wird. Ein wesentlicher Vorteil von dynamischen Bibliotheken gegenüber statischen ist, dass der ausführbare Code für alle Prozesse, die diese Bibliothek benutzen nur einmal in den Speicher geladen wird [15].

5.2.3 Fazit

Da der Agent nur als einzelner Prozess laufen soll, ist durch die Verwendung von dynamischen Bibliotheken kein wirklicher Vorteil gegeben. Es macht daher mehr Sinn die Bibliotheken der Google V8 Engine statisch in die ausführbare Datei zu linken, da man somit von externen Bibliotheken unabhängig bleibt.

5.3 Realisierungsart

Im letzten Abschnitt wurde erörtert, wie die V8-Bibliotheken als externes Modul in die eigene Komponente eingebunden werden sollen. Nun wird die Art der Umsetzung diskutiert. Es wird hier vor allem die Anforderung der Wiederverwendbarkeit betrachtet.

5.3.1 Dynamische Bibliothek

Die Umsetzung als dynamische Bibliothek wäre praktikabel, da dies ein einfaches Aktualisieren der Komponente ermöglichen kann. Wenn Teile der Implementierung aufgrund etwaiger Schwachstellen oder neuen Funktionen geändert werden, muss bei den Kundinnen und Kunden nicht jedes Programm, das die V8 Engine verwendet, einzeln aktualisiert werden, sondern nur die dynamische Bibliothek. Voraussetzung dafür ist, dass die Schnittstellen nicht geändert werden.

5.3.2 Statische Bibliothek

Kapselt man die Funktionalität innerhalb einer statischen Bibliothek, so erhält jede ausführbare Datei eine eigene Instanz. Dies bietet keine besonderen Vor- oder Nachteile.

5.3.3 Ausführbare Datei

Wird die komplette Funktionalität direkt innerhalb einer ausführbaren Datei umgesetzt, ist das Realisieren der Wiederverwendbarkeit nur schwer möglich. Eine Option wäre, die Ausführung von Javascriptdateien über eine RPC-Schnittstelle (Remote Procedure Call) zu ermöglichen. Dies zu implementieren würde jedoch einen unnötigen Mehraufwand verursachen. Außerdem besteht die Gefahr, dass der Prozess überlastet wird, wenn mehrere Teilnehmer gleichzeitig Zugriff auf die Schnittstelle verlangen.

5.3.4 Fazit

Die Umsetzung als ausführbare Datei mit einer RPC-Schnittstelle kann aufgrund der erläuterten Probleme als unpraktikabel angesehen werden und wird daher nicht in Be-

tracht gezogen. Eine Implementierung als dynamische Bibliothek setzt voraus, dass sich die Schnittstellen nicht ändert, was die Flexibilität in der Entwicklung neuer Funktionalitäten einschränken kann. Deshalb wird für die zu entwickelnde Programmkomponente das Erstellen einer statischen Bibliothek als Umsetzungskonzept gewählt. Folgende Grafik soll das Zusammenspiel der einzelnen Komponenten veranschaulichen:

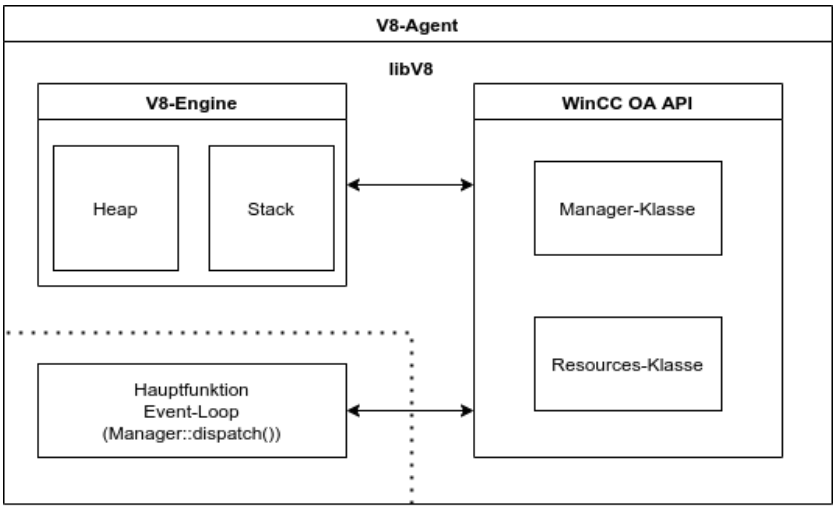


Abbildung 5.1: Dieses Übersichtsbild zeigt die statische Bibliothek *libV8* und wie sie verwendet wird.

Wie in Abbildung 5.1 gezeigt sind die WinCC OA und Google V8 Funktionen innerhalb der statischen Bibliothek *libV8* gekapselt. Die V8-Engine kümmert sich um das Ausführen von Javascriptdateien und die dazu benötigte Speicherverwaltung. Der Javascript-Sprachumfang wird über die WinCC OA Programmierschnittstelle um eigene Methoden erweitert. Diese Bibliothek kann dann innerhalb der Hauptfunktion eines Agenten verwendet werden, um die Ausführung von Javascriptdateien zu ermöglichen.

5.4 Handbuch für Integratorinnen und Integratoren

Javascript besitzt als Programmiersprache keine Typsicherheit, aus diesem Grund ist sie in größeren Codebasen nicht immer einfach handzuhaben. TypeScript bietet hier Abhilfe, da es ein typsicheres Programmieren von Javascript-Code ermöglicht. TypeScript-Quellcode kann dann durch einen TypeScript-Compiler in Javascript-Quellcode übersetzt werden. Damit man die innerhalb des V8-Agenten implementierten Funktionen in dieser Programmiersprache verwenden kann, müssen somit eigene Funktionsdeklarationen geschrieben werden. Um ein interaktives Handbuch für Systemintegratorinnen und Systemintegratoren zu erhalten, bietet sich hier die Literate-Programming Methodik an [12]. Die Idee ist es, die Funktionsdeklarationen direkt im Quellcode mit Hilfe von speziellen Quellcodekommentaren zu erklären. Später kann aus dem Programmcode und diesen Kommentaren ein Dokument erzeugt werden. Das de facto Standard Programm zum Generieren solcher Dokumentationen, *Doxygen*, unterstützt TypeScript und Javascript nur eingeschränkt [9]. Eine Alternative bietet daher *TypeDoc* [47].

5.4.1 TypeDoc

TypeDoc ist ein Programm zum Generieren von Dokumentationen aus einem TypeScript-Programmquellcode. Es ist möglich mit Hilfe von definierten Steuerbefehlen in den Kommentarblöcken Schnittstellen und Klassen zu beschreiben. Folgendes Beispiel soll zeigen, wie dies auszusehen hat:

Programm 5.1: TypeDoc Beispiel.

```
1  /**
2  * Returns the value stored within an datapoint element.
3  * @param dpe Datapoint element to get the value from.
4  * @returns The value stored within an datapoint element.
5  * Returned variable type depends on the datapoint element type.
6  *
7  * ### Basic usage example:
8  *
9  * ```ts
10 * var text = DpGet("System1:Example.text:_online_value");
11 * console.log(text); // :_online_value of System1:Example.text
12 * ```
13 */
14 declare function DpGet(dpe: string): any;
```

Wie in 5.1 ersichtlich, werden über die Anweisungen *@param* und *@return* die Funktionsparameter und der Rückgabewert dokumentiert. Außerdem kann beschreibender Beispielcode direkt bei der Funktionsdeklaration eingefügt werden.

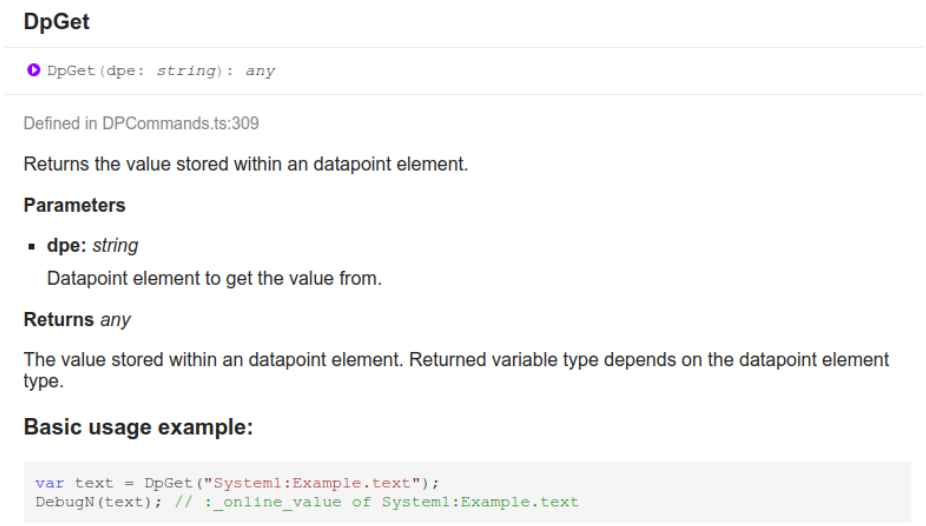


Abbildung 5.2: Generierte Dokumentation für *DpGet*.

In der Abbildung 5.2 ist die von *TypeDoc* erzeugte Dokumentation veranschaulicht. Parameter und Rückgabewert sind optisch hervorgehoben dargestellt und der Beispielquellcode wird mit Syntaxhervorhebung angezeigt.

5.4.2 Fazit

TypeDoc bietet eine gute Alternative zu *Doxygen* und wird deshalb für die Umsetzung des interaktiven Nachschlagewerks gewählt.

5.5 Konzeptioneller Beweis

Bevor mit der Implementierung eines eigenen Agenten gestartet wird, ist es zunächst sinnvoll, die Machbarkeit anhand einer einfachen konzeptionellen Umsetzung festzustellen. Dazu werden die aus den Beispielen 2.3.3 und 3.4.6 gewonnenen Erkenntnisse in einem eigenen Prototypen kombiniert. Das Ziel ist es die *dpConnect()*-Funktion der WinCC OA C++-Programmierschnittstelle an die Google V8 Javascriptengine anzubinden. Dazu muss zunächst eine eigene Observer-Klasse implementiert werden, die beim Erhalt von Datenpunkt-Wertänderungen eine Javascript-Methode aufrufen kann. Die aus den vorherigen Beispielen wiederverwendeten Klassen und Funktionen sind hier nicht mehr aufgeführt.

Listing 5.1: Observer-Klasse, erhält asynchrone Antworten und gibt diese an eine Javascript-Funktion weiter.

```
0  class DpObserver : public HotLinkWaitForAnswer {
1  public:
2    DpObserver(v8::Isolate* iso,
3      const v8::Persistent<v8::Function, v8::CopyablePersistentTraits<v8::Function>>&
       fun,
4      v8::Global<v8::Context>& ctx) :
5      m_Isolate(iso), m_Function(fun),
6      m_Context(std::move(ctx)) {};
7    // Diese Funktion wird bei jeder Wertänderung aufgerufen
8    void hotLinkCallBack(DpHLGroup &group) override {
9      std::cout << "- hotLinkCallBack(DpHLGroup), Execute JS Function -" << std::endl;
10     v8::Locker lock(m_Isolate); v8::Isolate::Scope isolate_scope(m_Isolate);
11     v8::HandleScope handle_scope(m_Isolate);
12     v8::Local<v8::Context> context = m_Context.Get(m_Isolate);
13     v8::Context::Scope context_scope(context);
14     // Marshalling von WinCC OA Datentypen nach V8 Javascript Datentypen
15     const int argc = 2;
16     v8::Local<v8::Value> argv[argc];
17     v8::Local<v8::Array> dpeArr = v8::Array::New(m_Isolate);
18     v8::Local<v8::Array> varArr = v8::Array::New(m_Isolate);
19     uint32_t num = 0;
20     for (DpVCItem* item = group.getFirstItem(); item; item = group.getNextItem()) {
21       Variable* value = item->getValuePtr(); // Neuer Wert
22       CharString dpe = item->getDpIdentifier().toString(); // Name des DPElement
23       dpeArr->Set(num, v8::String::NewFromUtf8(m_Isolate, dpe));
24       if(value->isA(VariableType::BIT_VAR) == VariableType::BIT_VAR){
25         varArr->Set(num, v8::Boolean::New(m_Isolate, static_cast<BitVar*>(value)->
       getValue()));
26       } else if (value->isA(VariableType::FLOAT_VAR) == VariableType::FLOAT_VAR) {
27         varArr->Set(num, v8::Number::New(m_Isolate, static_cast<FloatVar*>(value)->
       getValue()));
28       } else {
29         varArr->Set(num, v8::Null(m_Isolate));
30       }
```

```
31        num += 1;
32      }
33      argv[0] = dpeArr; argv[1] = varArr;
34      v8::TryCatch tryCatch(m_Isolate);
35      v8::Local<v8::Function> f = v8::Local<v8::Function>::New(m_Isolate, m_Function);
36      // Falls der Javascript Functionsscope (in js) über .bind gesetzt wurde verwende
37      // diesen, ansonsten verwende das global Objekt des Ausführungskontext
38      v8::Local<v8::Object> fScope = f->GetBoundFunction().As<v8::Object>();
39      if(fScope == v8::Undefined(m_Isolate)) fScope = context->Global();
40      // rufe JS Funktion auf, übergebe dpes als param1 und werte als param2
41      f->Call(Scope, argc, argv);
42      if (tryCatch.HasCaught()) ReportException(m_Isolate, &tryCatch);
43    }
44    void hotLinkCallBack(DpMsgAnswer &answer) override {}  // Stub, nicht verwendet
45  private:
46    v8::Isolate* m_Isolate;
47    v8::Persistent<v8::Function, v8::CopyablePersistentTraits<v8::Function>>
        m_Function;
48    v8::Global<v8::Context> m_Context;
49  };
```

In dem Codeabschnitt 5.1 kann man erkennen, dass die Javascript-Funktion dem Konstruktor als Persistent-Handle übergeben wird. Somit gibt der Garbage-Collector das dahinterliegende Objekt nicht frei. Der Destruktor der DpObserver-Klasse müsste die *SetWeak()*-Funktion des *m_Function*-Objekts aufrufen, was hier aber aus Platzgründen nicht ausimplementiert wurde. Außerdem wird ein *Isolate*- und ein *Context*-Objekt übergeben um später die Context-Ausführungsumgebung angeben zu können, die für die Ausführung der Javascript-Funktion verwendet werden soll. Dies geschieht nach Aufrufen von *hotlinkCallBack(DpHLGroup)* am Anfang der Funktion. Danach muss ein Marshalling der WinCC OA Datentypen nach Javascript erfolgen. Der Begriff Marshalling bedeutet umwandeln von Daten in ein Format, dass die Übermittlung an eine prozessfremde Schnittstelle ermöglicht [6]. Hierbei werden zwei Javascript-Arrays erstellt. Das Erste erhält die Namen der Datenpunktelemente und das Zweite die Werte. Diese werden für die Übergabe in dem *argv*-Objekt abgelegt (Zeile 33). Über ein *TryCatch*-Objekt ist ein Auffangen eventueller Fehler in der aufzurufenden Funktion realisiert. In Javascript ist es möglich über die *.bind* Anweisung einer Funktion einen eigenen Scope zuzuweisen. Dies wird in Zeile 38 geprüft und in dem Fall, dass ein Funktionsscope gesetzt wurde, wird dieser für den Aufruf verwendet. Zuletzt wird die Javascript-Funktion aufgerufen und etwaige Fehler ausgegeben. Die Implementierung der *ReportException()*-Funktion kann in Anhang C eingesehen werden. Im nächsten Schritt muss der Javascript-Sprachumfang um die *dpConnect()*-Funktion erweitert werden:

Listing 5.2: C++-Code, ermöglicht *DpConnect* via Javascript.

```
0 void DpConnect(const v8::FunctionCallbackInfo<v8::Value>& args) {
1   v8::Isolate* isolate = args.GetIsolate();
2   v8::Isolate::Scope iscope(isolate);
3   v8::HandleScope hscope(isolate);
4   if (args.Length() == 2 && args[0]->IsString() && args[1]->IsFunction()) {
5     v8::String::Utf8Value param1(args[0]->ToString());
6     CharString str(*param1); // Marshalling V8 -> WinCC OA
7     DpIdentifier dpe;
8     Agent::getId(str, dpe);
```

```
9       // JS Callback-Funktion als Persistente Variable (->Schutz vor Garbage Collector)
10      v8::Persistent<v8::Function, v8::CopyablePersistentTraits<v8::Function>>
        callback;
11      callback.Reset(isolate, args[1].As<v8::Function>());
12      // Ausführungscontext als Globales V8 Objekt, wird benötigt
13      // um diesen context zu einem späteren Zeitpunkt wieder verwenden
14      // zu können.
15      v8::Global<v8::Context> context;
16      context.Reset(isolate, isolate->GetCurrentContext());
17      // registriere erhalt von Wertänderungen, bei änderungen soll
18      // die Javascript Funktion aufgerufen werden.
19      bool ret = Agent::dpConnect(dpe, new DpObserver(isolate, callback, context),
        true);
20      args.GetReturnValue().Set(v8::Boolean::New(isolate, ret)); // Javascript Rü
        ckgabewert
21    }
22    else { // Werfe exception
23      isolate->ThrowException(v8::Exception::TypeError(v8::String::NewFromUtf8(isolate
        , "Invalid parameters, Expected: string dpe, function callback")));
24    }
25  }
```

Die Funktion 5.2 überprüft zuerst, ob der aufrufende Javascript-Code die passenden Objekte übergeben hat, ansonsten wird eine Exception geworfen die Auskunft über die erwarteten Parameter gibt. Danach wird über *Agent::getId()* die textuelle Repräsentation des Datenpunktelements in den benötigten *DpIdentifier* Datentyp umgewandelt. Zuletzt wird die WinCC OA *dpConnect* Funktion mit der vorher beschriebenen *Observer*-Klasse als Parameter aufgerufen. Dem Observer werden das *Isolate*-Objekt, die Javascript-Callbackfunktion und der Ausführungskontext im Konstruktor mitgegeben. Diese Funktion kann man genauso wie *PrintParams* über das *global*-Objekt beim Starten des Programmes an das Context-Objekt übergeben:

Listing 5.3: Hinzufügen der *DpConnect()*-Funktion zum *ObjectTemplate*-Objekt

```
global->Set(v8::String::NewFromUtf8(isolate, "DpConnect", v8::NewStringType::kNormal
    ).ToLocalChecked(), v8::FunctionTemplate::New(isolate, DpConnect));
```

Initialisiert man nun in der Hauptfunktion des Programms die WinCC OA *Resources*-Klasse, erstellt ein *Agent*-Objekt und richtet die Google V8 Engine, wie in den zuvor gezeigten Beispielen ein, kann nach erreichen der Hauptschleife mit dem *dispatch()* Aufruf die *DpConnect*-Funktion in Javascript aufgerufen werden:

Listing 5.4: Beispielaufruf der *DpConnect()*-Funktion.

```
1 // Beispiel Javascript, ausgeführt von prototyp3
2 var i = 0;
3 DpConnect("System1:ExampleDP_DDE.f1:_online.._value", function(dpes, vars){
4     PrintParams(dpes, vars, i);
5     i++;
6 })
```

Das in 5.4 gezeigte Beispiel registriert sich auf Wertänderungen und gibt diese zusammen mit einem Zähler aus. Bei schreiben der Werte '47.11', '42', '47,11' und '42' auf das Datenpunktelement 'System1:ExampleDP_DDE.f1' wird folgende Ausgabe generiert:

Listing 5.5: Ausgabe bei Wertänderungen.

```
prototyp342:- hotLinkCallBack(DpHLGroup), Execute JS Function -
prototyp342:System1:ExampleDP_DDE.f1:_online.._value
prototyp342:47.11
prototyp342:0
prototyp342:- hotLinkCallBack(DpHLGroup), Execute JS Function -
prototyp342:System1:ExampleDP_DDE.f1:_online.._value
prototyp342:42
prototyp342:1
prototyp342:- hotLinkCallBack(DpHLGroup), Execute JS Function -
prototyp342:System1:ExampleDP_DDE.f1:_online.._value
prototyp342:47.11
prototyp342:2
prototyp342:- hotLinkCallBack(DpHLGroup), Execute JS Function -
prototyp342:System1:ExampleDP_DDE.f1:_online.._value
prototyp342:42
prototyp342:3
```

Wie in 5.5 ersichtlich, werden die Wertänderungen und die Zählervariable ausgegeben. Auch dieses Beispiel befindet sich als CMake [30] Projekt auf der CD-ROM/DVD und ist unter *Thesis/prototype3* zu finden. Es kann mit folgenden Befehlen übersetzt werden. Dazu wird das *SMF_3P* Verzeichnis und eine WinCC OA Installation benötigt.

SUSE Linux Enterprise 12.5:

```
1 $ mkdir build
2 $ cd ./build
3 $ cmake .. -DSMF_3P=~/SMF_3P -DWINCCOA_INSTALL=C:/Siemens/Automation/WinCC_OA/3.16
4 $ cmake --build .
```

Windows 10:

```
1 $ mkdir build
2 $ cd ./build
3 $ cmake .. -T v140 -A x64 -DSMF_3P=C:/SMF_3P -DWINCCOA_INSTALL=C:/Siemens/Automation
      /WinCC_OA/3.16
4 $ cmake --build .
```

Kapitel 6

Entwicklung des Prototypen

In diesem Kapitel wird die Implementierung des SCADA-Agenten anhand des im letzten Kapitel aufgezeigten Konzeptes erläutert.

6.1 Übersetzten des Google V8 C++ Quellcodes

Dieser Abschnitt beschreibt wie der Google V8 Quellcode in eine statische Bibliothek übersetzt werden kann.

6.1.1 Linux

Aufgrund der Compilervorgaben von WinCC OA gestaltet sich die Übersetzung des Quellcodes auf Linux schwierig. Das Google V8 Buildsystem ist auf Debian-basierende Distributionen ausgelegt. Das bedeutet die Engine kann auf dem Zielsystem SUSE Linux Enterprise 12.5 nicht übersetzt werden. Ein Alternativsystem mit der passenden Compilerversion stellt Ubuntu 18.04 zur Verfügung. Die Bibliothek kann auf diesem System kompiliert und danach auf SUSE Linux Enterprise 12.5 verwendet werden.

Programm 6.1: Installation von GCC-4.8.5 auf Ubuntu 18.04

```
1 $ sudo apt update
2 $ sudo apt upgrade
3 $ sudo apt install git libglib2.0-dev python g++-4.8 gcc-4.8
4 $ sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-4.8 10
5 $ sudo update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-4.8 10
```

In dem Beispiel 6.1 ist gezeigt wie auf einem Ubuntu 18.04 System der benötigte Compiler nachinstalliert und als Standardoption zum Übersetzten von Quellcode konfiguriert werden kann. Hat man diese Schritte durchgeführt, kann man die Google V8 Engine mit dem in Anhang A gezeigten Skript kompilieren. Dabei werden zuerst automatisch der Quellcode und alle benötigten Dateien aus dem Google-Versionskontrollsystem bezogen. Als Nächstes wird eine Buildkonfiguration erzeugt. In dieser wird das Buildsystem angewiesen eine statische Bibliothek zu generieren. Danach erfolgt eine Nachbearbeitung

der generierten Dateien, weil die Standardkonfiguration einen neueren Compiler voraussetzt. Hierbei wird die *-std=gnu++14* Anweisung durch *-std=gnu++11* ersetzt. Zum Schluss wird der Buildprozess gestartet. Die fertigen Kompilate sind unter *v8* und die Headerdateien unter *include* zu finden und können auf einem SUSE Linux Enterprise 12.5 System verwendet werden.

6.1.2 Windows

Auf Windows gestaltet sich dieser Prozess deutlich einfacher. Hier ist es zunächst nötig, Visual Studio 2015 zu installieren. Dadurch wird das *Visual Studio Toolset* v140 zur Verfügung gestellt. Im nächsten Schritt muss man das in Anhang A gezeigte Powershell-Skript für Windows mit folgendem Kommandozeilenparameter ausführen:

```
$ powershell.exe -noprofile -executionpolicy bypass -file .\build_windows.ps1
```

Dadurch wird ähnlich wie unter Linux der Quellcode und weitere Dateien aus dem Google-Versionskontrollsystem bezogen, die Konfigurationsdateien erzeugt und der Kompiliervorgang angestoßen. Die übersetzten Bibliotheksdateien und die dazugehörigen Headerdateien befinden sich danach unter *v8* und *include*.

6.2 Implementierung der Programmbibliothek

Da erklärt wurde, wie man ein Kompilat der Google V8 Engine als statische Bibliothek erhält, kann auf die Implementierung der Programmkomponente eingegangen werden. Hierbei werden die einzelnen Klassen anhand deren UML-Repräsentation (*Unified Modeling Language*) beschrieben. Das gesamte UML-Diagramm mit allen Relationen kann in Anhang D eingesehen werden.

6.2.1 Implementierung der Singleton-Klasse

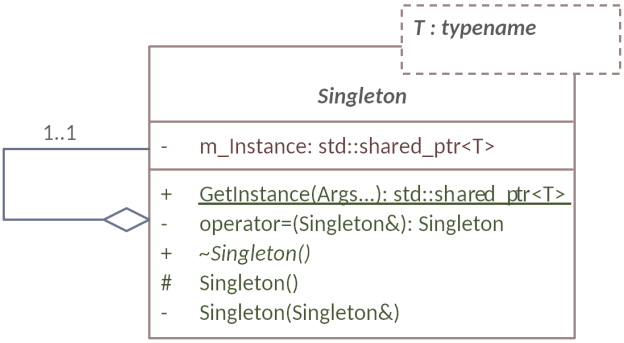


Abbildung 6.1: Singleton Template-Klasse.

Diese in 6.1 gezeigte Template-Klasse implementiert eine wiederverwendbare Umsetzung des Singleton-Entwurfsmusters [13]. Ableitende Klassen erhalten, indem sie sich selbst als Template-Parameter übergeben, eine *GetInstance()*-Funktion, die die einzige

Instanz als *Shared-Pointer* zurückgibt. Dadurch ist ein korrektes Freigeben des Speichers am Ende der Programmausführung gewährleistet. *Shared-Pointer* wurde *Unique-Pointer* vorgezogen, da bei Bedarf der Zeitpunkt der Speicherfreigabe beeinflusst werden kann. Ableitende Klassen müssen zusätzlich deren Konstruktor als *protected* deklarieren und die Singleton-Klasse als *friend* hinzufügen. Die *GetInstance()*-Methode nimmt jene Parameter entgegen, die eine ableitende Klasse zur Konstruktion benötigt.

6.2.2 Implementierung der libV8-Klasse

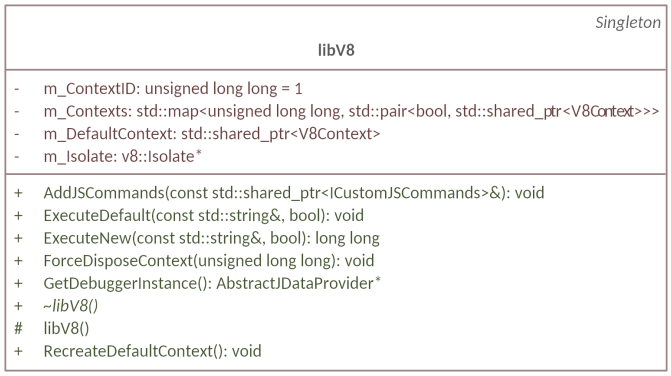


Abbildung 6.2: libV8-Singleton-Klasse.

Abbildung 6.2 zeigt die Hauptkomponente der Bibliothek. Diese Klasse verwaltet das *Isolate*-Objekt der Google V8 Engine und alle Ausführungsumgebungen. Über die Funktionen *ExecuteDefault()* und *ExecuteNew()* ist es möglich Javascriptdateien auszuführen. Beide akzeptieren als ersten Parameter einen Pfad zu einer Skriptdatei und als zweiten einen booleschen Wert der angibt, ob der Ausführungskontext am Ende des Skriptes aufgeräumt werden soll oder nicht. Erstere verwendet immer dieselbe Ausführungsumgebung (*m_DefaultContext*). Dadurch ist es möglich eine Liste von Skripten auszuführen, die auf die gleichen Ressourcen Zugriff besitzen. Zweitere erstellt für jeden Aufruf ein eigenes *Context*-Objekt und gibt eine Nummer zurück, die diesen Kontext identifiziert. Somit lassen sich mehrere Skripte unabhängig voneinander ausführen. Die Methoden *RecreateDefaultContext()* und *ForceDisposeContext()* erlauben ein späteres Aufräumen des Standardkontexts und der neu erstellten. Zweiteren muss man die über *ExecuteNew()* erhaltene Identifikationsnummer der zu löschenden Ausführungsumgebung als Parameter mitgeben. Mit Hilfe der *AddJSCommands()*-Funktion kann man den Ausführungskontexten eigene Javascriptfunktionen zur Verfügung stellen. Über diesen Mechanismus ist ein dynamisches Erweitern möglich. *GetDebuggerInstance()* gibt ein abstraktes Observer-Objekt retour, das einer Websocket-Server Implementierung des *SiPlant* Produktes übergeben werden kann. Da innerhalb eines Programms nur eine einzige Google V8 Engine gestartet werden darf, leitet diese Klasse von der Singleton-Template-Klasse ab. Dadurch ist ein mehrfaches Instanziieren ausgeschlossen.

6.2.3 Implementierung der SIS_InspectorClient-Klasse



Abbildung 6.3: SIS_InspectorClient-Klasse.

Diese Klasse implementiert, wie in Abbildung 6.3 erkennbar, drei Interfaces. Das *AbstractJDataProvider*-Interface ist eine Observer-Klasse für den in dem *SiPlant* Produkt verwendeten Websocket-Server. Werden Nachrichten erhalten, gibt der Server diese an die *ExecuteCommand()*-Methode weiter. Der erste Parameter ist die Nachricht im JSON-Format und die zweite der Websocket-Endpunkt. Die Implementierung dieser Funktion gibt eine leere Struktur zurück, wodurch der Websocket-Server keine unmittelbare Antwort an den Client sendet. Über das *SIS_IEndpoint*-Objekt ist es später jederzeit möglich Nachrichten an diesen zu übermitteln. Darum wird es als *m_Client*-Member-Variable gespeichert. Die *GetWSClient()*-Funktion ermöglicht Zugriff auf diese Variable. Da man in der Regel nur ein *Isolate* benötigt, leitet diese Klasse ebenfalls von der Singleton-Template-Klasse ab. Dank der geerbten *GetInstance()*-Funktion kann so das zu verwendende *Isolate*-Objekt an den geschützten Konstruktor übergeben werden. Dies geschieht beim Aufrufen der *GetDebuggerInstance()*-Methode des *libV8*-Singletons automatisch. Des Weiteren wird im Konstruktor ein *V8Inspector*-Objekt mit einer eigenen *V8InspectorSession* erstellt. Dieser Session wiederum wird mit einer Instanz der *SIS_InspectorFrontend* Klasse zum Senden von Antworten der V8 Engine ausgestattet. Über die *AddContext()*- und *RemoveContext()*-Funktionen kann man zu überwachende Ausführungsumgebungen hinzufügen und entfernen. Schließt ein Websocket-Endpunkt die Verbindung, führt dies durch den Websocket-Server zu einer Ausführung der *OnConnectionClosed()*-Methode. In dieser wird die *m_Client*-Member-Variable auf nullptr gesetzt, damit das *SIS_InspectorFrontend*-Objekt keine Antworten mehr sendet. Das *v8InspectorClient*-Interface schreibt die Umsetzung der Methoden *runMessageLoopOnPause()* und *quitMessageLoopOnPause()* vor. Wird ein Javascript zum Beispiel wegen eines Breakpoints gestoppt, arbeitet die Erstere so lange alle Websocket-Nachrichten ab, bis die zweitere Funktion aufgerufen wird um die Ausführung fortzusetzen.

6.2.4 Implementierung der SIS_InspectorFrontend-Klasse

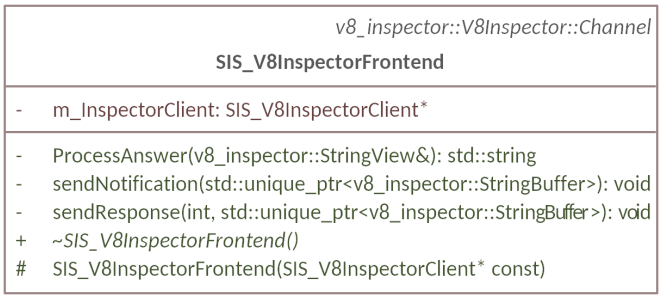


Abbildung 6.4: Implementierung des *v8Inspector::Channel*-Interfaces.

Die Google V8 Engine antwortet auf Debugging-Befehle über das *v8Inspector::Channel*-Interface. Um an die Browser-Entwicklungswerkzeuge Antworten auf Analysebefehle senden zu können wurde daher die in 6.4 gezeigte Klasse realisiert. Der Konstruktor kann über eine Friend-Beziehung nur von dem *SIS_InspectorClient*-Singleton aufgerufen werden. Dieses übergibt sich beim Erzeugen dieser Klasse selbst. Innerhalb der *sendNotification()* und *sendResponse()* Methoden werden die von der V8 Engine erhaltenen Nachrichten an den in der *SIS_InspectorClient*-Klasse gespeicherten Websocketendpunkt gesendet. *ProcessAnswer()* ist eine Hilfsfunktion, die die Antworten der V8 Engine in normale Strings umwandelt.

6.2.5 Implementierung der V8Context-Klasse

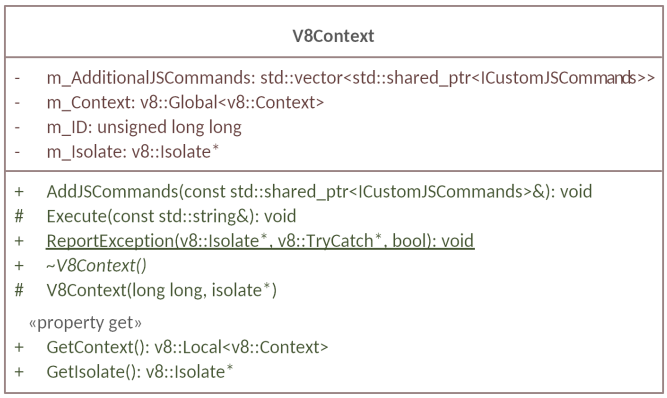


Abbildung 6.5: V8Context-Klasse, repäsentiert einen Ausführungskontext.

Die in Abbildung 6.5 gezeigte Klasse verwaltet einzelne Ausführungskontexte und ist somit für das Ausführen der Javascriptdateien zuständig. Um eine falsche Verwendung zu vermeiden, ist der Konstruktor nur von der *libV8*-Klasse aufrufbar (Friend-Beziehung). Beim Erstellen einer *V8Context*-Instanz wird die Identifikationsnummer mitübergeben.

Des Weiteren wird im Konstruktor die *m_AdditionalJSCommands* Variable mit Standardbefehlen für den Datenbankzugriff und die Alarmbehandlung befüllt. Das *libV8*-Singleton gibt die über *ExecuteDefault()* oder *ExecuteNew()* erhaltenen Dateipfade an die *Execute()*-Methode eines jeweiligen *V8Context*-Objekts weiter. Bevor das Skript gestartet wird, übergibt die Klasse das *Context*-Objekt an das *SIS_V8InspectorClient*-Singleton. Dadurch wird eine Überwachung der Ausführungsumgebung zur Fehleranalyse über Browser-Entwicklungswerkzeuge ermöglicht. Als Nächstes werden die *RegisterCommands()*- und *RegisterToGlobal()*-Funktionen aller *ICustomJSCommands*-Objekte der *m_AdditionalJSCommands*-Member-Variable aufgerufen. Wird der Ausführungskontext zerstört, so werden im Destruktor alle *FreeResources()*-Methoden dieser Objekte aufgerufen und die Überwachung über die *RemoveContext()*-Funktion des *SIS_InspectorClient*-Singletons beendet.

6.2.6 Implementierung des ICustomJSCommands-Interface

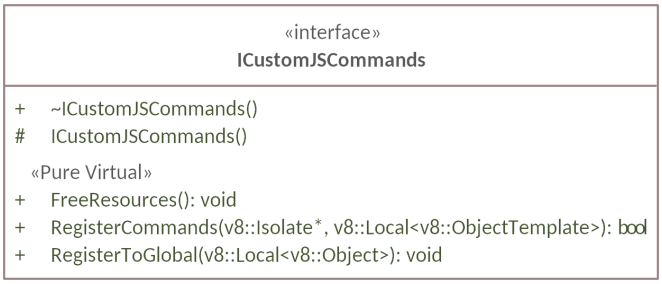


Abbildung 6.6: ICustomJSCommands-Interface zum Erweitern des Javascript-Sprachumfangs.

Das Interface in der Grafik 6.6 erlaubt ein Erweitern des Javascript-Sprachumfangs. Konkretisierungen müssen drei Funktionen implementieren, die von der *V8Context*-Klasse aufgerufen werden.

- *RegisterCommands()* ermöglicht das Hinzufügen von C++-Methoden zu dem *ObjektTemplate*-Objekt bevor das *Context*-Objekt erstellt wird.
- Nach Konstruieren des Ausführungskontexts, wird *RegisterToGlobal()* aufgerufen. Dieser Funktion wird die auf Basis des *ObjektTemplate*-Objekts erzeugte Instanz mitgegeben. Dadurch ist es über C++ möglich Javascript-Methoden auszuführen und Variablen zu verändern.
- *FreeResources()* wird beim Freigeben der Ausführungsumgebung aufgerufen um allokierte Ressourcen freigeben zu können.

Konkretisierung DPJSCommands

Diese Konkretisierung stellt den Ausführungsumgebungen, wie in der Grafik 6.7 ersichtlich, Zugriffsmethoden auf den Datenhaushalt des SCADA-Systems zur Verfügung. Die rekursiven Funktionen *ConvertToJSValue()* und *ConvertToSVariable()* beinhalten If-

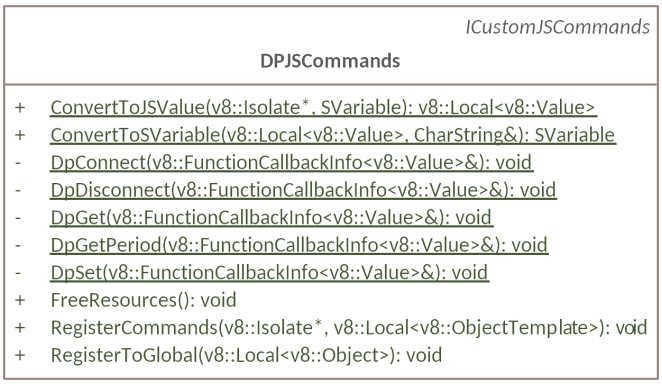


Abbildung 6.7: Funktionen für den Datenbankzugriff.

Else-Kaskaden für das Marshalling von WinCC OA Datentypen nach Javascript und vice versa. Die Rekursion wird benötigt um mit Arrays umgehen zu können.

Konkretisierung AlertJSCommands

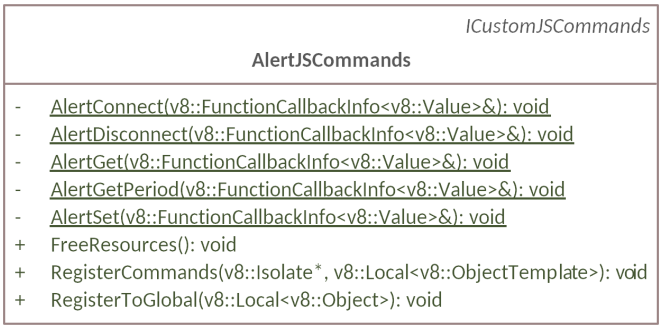


Abbildung 6.8: Funktionen für den Alarmzugriff.

Analog zu *DPJSCommands* wird über die in Abbildung 6.8 erkennbare Klasse der Zugriff auf die Alarmfunktionen ermöglicht. Marshalling zwischen WinCC OA- und Javascript Datentypen wird über die im letzten Abschnitt beschriebenen Methoden *ConvertToJSValue()* und *ConvertToSVariable()* bewerkstelligt.

6.3 Implementierung des Agenten

Der Agent verwendet die zuvor beschriebene Bibliothek, um Javascriptdateien ausführen zu können. Die Datei ist über einen Kommandozeilenparameter übergebbar. In der Hauptfunktion ist am Anfang das Auslesen des Skriptpfades aus den übergebenen Parametern implementiert. Als Nächstes wird die Verbindung mit dem SCADA-System hergestellt. Darauf folgt das Starten des *SiPlant* Websocket-Servers. Dieser erhält das

von der *GetDebuggerInstance()*-Funktion zurückgegebenen Observer-Objekt. Dadurch ist ein Verbinden mit einer Browser-Entwicklerkonsole möglich. Danach wird vor der Hauptschleife mit dem wiederkehrenden *dispatch()* Aufruf das übergebene Skript an die *ExecuteDefault()*-Funktion des *libV8*-Singletons weitergereicht. Der boolesche Parameter ist hier auf 'false' gesetzt, da der Ausführungskontext nicht zerstört werden soll. Somit ist es möglich, eigene Algorithmen zu implementieren und eventorientiert Berechnungen in Javascript durchzuführen.

Kapitel 7

Praxiserfahrungen

Auf Basis des im vorhergehenden Kapitel 6 entworfenen Prototypen wurde für das Produkt *SiPlant* ein Agent mit erweitertem Funktionsumfang entwickelt. Der Quellcode kann in der beiliegenden CD-ROM/DVD eingesehen werden. Diese Entwicklungen finden bereits in einigen Projekten Einsatz die in diesem Kapitel vorgestellt werden.

7.1 BMW Gruppe, Dingolfing Deutschland

Für die BMW Gruppe wurde *SiPlant* in zwei Batteriemodulfertigungsanlagen integriert. Als Kundenanforderung wurde von dem Systemintegrator unter Zuhilfenahme des V8-Agenten eine Schrittkettenanalyse mit Wiedergabefunktion zum Überwachen und Optimieren der Transitionen umgesetzt. Zusätzlich konnte das Anbinden folgender Teilnehmer in der *Feldebene* erleichtert werden:

- Anlage MP05: 21 S7Plus Steuerungen mit 98000 Eingabe- und Ausgabekanälen.
- Anlage MP02: 29 S7-300 Steuerungen mit 80000 Eingabe- und Ausgabekanälen.

Laut Kundenrückmeldung konnte mit der Schrittkettenanalyse die Produktion um 9% und die Anlagenverfügbarkeit um 15% gesteigert werden.

7.2 Daimler AG, Factory 56 Sindelfingen Deutschland

In diesem Projekt wird die *SiPlant*-Erweiterung *SIMOVE* für die Steuerung von 507 fahrerlosen Transportfahrzeugen in Produktion und Logistik der Mercedes Benz S-Klassen Limousine verwendet. Die Fahraufträge werden von der *Produktionsleitebene* an das SCADA-System über das Daimler-Integraprotokoll übermittelt. Mit Hilfe der angebundenen Javascriptengine wurde hier ein Auswertungswerkzeug für historische Integra Protokolldaten entwickelt.

7.3 Valmet, Salo Finnland

Valmet ist ein Zulieferbetrieb von Hochvoltspeicher für die Daimler AG. Es werden dort drei Produktionslinien mit 119 S7Plus Steuerungen in *SiPlant* visualisiert. Der V8-Agent

fand hier als Hilfswerkzeug für den Import von 180000 Eingabe- und Ausgabekanälen Einsatz.

7.4 Geely, Ningbo China

Für Geely wurde in einem Pilotprojekt *SiPlant* als SCADA-System für eine Prototypenfertigung aufgesetzt. Insgesamt sind in diesem System 63000 S7Plus und OPC-UA Eingabe- und Ausgabekanäle angebunden. Bei dem Import konnte der entwickelte Javascript-Agent Unterstützung leisten.

7.5 Fazit

Die in dieser Arbeit entwickelte Programmkomponente konnte sich wie oben angeführt bereits in der Praxis als nützlich erweisen. Dennoch ist das Umsetzungskonzept nicht optimal, desswegen wird im nächsten Abschnitt auf die Nachteile eingegangen.

7.5.1 Nachteile

Da die Google V8 Javascriptengine nur den von ECMA standardisierten ECMAScript Sprachumfang implementiert [33], fehlen viele der von den Browsern unterstützten und von W3C definierten Web-APIs [39]. Dadurch kann man nur eingeschränkt auf Drittanbieterbibliotheken zurückgreifen, weil diese oft von den Web-APIs abhängig sind.

Kapitel 8

Zusammenfassung und Ausblick

Im Rahmen dieser Arbeit wurde ein SCADA-Agent zum Ausführen von Javascriptcode sowohl konzipiert als auch implementiert. Ausgehend von den Anforderungen der Systemintegratorinnen und Systemintegratoren der Abteilung *DI FA SO SIS* innerhalb des Siemens-Konzerns wurde durch Betrachten verschiedener Umsetzungsmöglichkeiten ein Konzept für eine individuelle Softwarelösung erarbeitet. Während der Entwicklung ist ein Werkzeug auf Basis der WinCC OA- und Google V8-Programmierschnittstellen entstanden, dass eine einfachere Inbetriebnahme und Individualisierung des Produktes *SiPlant* ermöglicht.

Während der folgende Abschnitt dieses Kapitels die Ergebnisse dieser Masterarbeit zusammenfasst, folgt im letzten Unterkapitel ein Ausblick auf mögliche Verbesserungen.

8.1 Zusammenfassung

In Kapitel 2 wurde zunächst der Begriff Multi-Agent-SCADA-System auf Basis bestehender Definitionen beschrieben. Dadurch konnte erkannt werden, dass das von ETM Professional Control entwickelte *Windows Control Center Open Architecture* die Eigenschaften eines solchen Systems besitzt. Um zu verstehen, wie man dieses Produkt um eigene Agenten erweitern kann, wurde anhand eines Beispiels die Funktionsweise der Programmierschnittstelle erklärt. Das Kapitel 3 widmete sich der Google V8 Javascriptengine. Hier wurden die Grundlagen der Compilertheorie umrissen um darauf aufbauend die Mechanismen dieser Programmkomponente veranschaulichen zu können. Im letzten Abschnitt wurde gezeigt, wie man diese Engine in eigenen Entwicklungen verwenden kann, um Javascriptcode ausführen zu können. Im darauffolgenden Kapitel 4 wurden die Anforderungen an den Agenten aufgrund der zu lösenden Probleme bei der Anlagenintegration von Multi-Agent-SCADA-Systemen definiert. Hier wurde vor allem die Sichtweise der Systemintegratorinnen und Systemintegratoren in Betracht gezogen. Ein detailliertes Umsetzungskonzept konnte in Kapitel 5 durch implementieren eines konzeptuellen Beweises erarbeitet werden. In Kapitel 6 wurde auf Basis dieses Beweises ein eigener Agent mit erweitertem Funktionsumfang umgesetzt, um die in Kapitel 4 definierten Anforderungen zu erfüllen. Dadurch wurde ein Werkzeug entwickelt, dass die dynamisch erweiterbare Skriptsprache Javascript ausführen kann und erwei-

terte Debugging-Funktionen unterstützt. Zuletzt wurde die Nützlichkeit dieses Agenten anhand von abgeschlossenen Projekten in Kapitel 7 aufgezeigt.

8.2 Ausblick

Obwohl die entwickelte Programmkomponente bereits eine gute Hilfestellung bei der Projektintegration und Projektindividualisierung leisten konnte, ist aufgrund der fehlenden Unterstützung der von W3C standardisierten Web-APIs die gewünschte Flexibilität nur eingeschränkt gegeben. Außerdem steht das *SiPlant* Produkt aufgrund von Lizenzverkäufen und Hinzunahme von externen Ressourcen im Projektgeschäft einer stetig wachsenden Anzahl von Systemintegratorinnen und Systemintegratoren mit unterschiedlichen technischen Ausbildungen zur Verfügung. Daher wäre es vom Vorteil, anstelle eines dezidierten V8-Agenten eine Softwarebibliothek zu entwerfen, die von einer Vielzahl von Programmiersprachen verwendet werden kann, um auf das proprietäre WinCC OA SCADA-System zugreifen zu können. Technisch wäre dies über eine Fremdfunktionsschnittstelle umsetzbar [7]. Die meisten Programmiersprachen besitzen eine Möglichkeit, dynamische Softwarebibliotheken (Dynamic-Link Libraries oder Shared Objects) mit einer C-Programmierschnittstelle zur Laufzeit nachzuladen. Das Marshalling der Datentypen muss hierbei in der Zielsprache erfolgen.

Für Javascript existiert auf Basis der Google V8 Engine die plattformübergreifende Laufzeitumgebung NodeJS. Mit Hilfe eines eigenen Paketmanagers ist ein flexibles Nachladen von Drittanbieterbibliotheken möglicht, somit ist eine gute Erweiterbarkeit gegeben [43]. Außerdem unterstützt NodeJS die W3C Web-APIs. Über spezielle NodeJS-Bibliotheken ist ein Anbinden von Fremdfunktionsschnittstellen realisierbar. Somit kann man eigenen nativen Code der Javascript-Laufzeitumgebung zur Verfügung stellen. Dies soll im folgenden Beispiel gezeigt werden:

8.2.1 Fremdfunktionsschnittstelle (FFI)

Listing 8.1: Header-Datei des Bibliothekcodes.

```
0 #ifndef PROTOTYP4_H
1 #define PROTOTYP4_H
2 #ifdef _WIN32
3 #ifdef _BUILDING
4 #define DLLEXPNOMANGLE extern "C" __declspec( dllexport )
5 #else
6 #define DLLEXPNOMANGLE extern "C" __declspec( dllimport )
7 #endif
8 #else
9 #define DLLEXPNOMANGLE extern "C"
10 #endif
11 // Erzeugt eine neue Instanz
12 DLLEXPNOMANGLE void PROTO_New(void*& instance);
13 // Führe Funktion von Instanz aus
14 DLLEXPNOMANGLE void PROTO_Greet(void* inst, char* msg);
15 // Lösche Instanz
16 DLLEXPNOMANGLE void PROTO_Delete(void*& instance);
17 #endif // PROTOTYP4_H
```

In den Funktionsdefinitionen 8.1 ist erkennbar, dass nur statische Datentypen verwendet werden und der C++-Compiler angewiesen wird keine Namensdekorationen zu erzeugen.

Listing 8.2: Implementierung des Bibliothekcodes.

```
0 #include "prototyp4.h"
1 #include <iostream>
2 #include <string>
3 class Greeter {
4 public:
5   void Welcome(const std::string& name){
6     std::cout  << "Hello, "<< name << std::endl;
7   };
8 };
9 DLLEXPNOMANGLE void PROTO_New(void*& inst){
10   inst = new Greeter();
11 }
12 DLLEXPNOMANGLE void PROTO_Greet(void* inst, char* msg){
13   Greeter* obj = static_cast<Greeter*>(inst);
14   obj->Welcome(msg);
15 }
16 DLLEXPNOMANGLE void PROTO_Delete(void*& inst){
17   Greeter* obj = static_cast<Greeter*>(inst);
18   delete obj; inst = nullptr;
19 }
```

Die Implementierung 8.2 zeigt, wie der komplexe Datentyp *Greeter* über die exportierten Funktionen übergeben werden kann.

Listing 8.3: Skript node.js, lädt Bibliothek und führt Funktionen aus.

```
1 var ffi = require('ffi-napi');
2 var ref = require('ref-napi');
3 // Lade je nach Plattform .dll oder .so
4 var libpath = "./libprototyp4.so"
5 if(process.platform === "win32") {
6   libpath = "./prototyp4.dll";
7 }
8 // Marshalling der Funktionen/Datentypen
9 var lib = ffi.Library(libpath, {
10     'PROTO_New': [ 'void', [ ref.refType('void*') ] ],
11     'PROTO_Greet': [ 'void', [ 'void*', 'string' ] ],
12     'PROTO_Delete': [ 'void', [ref.refType('void*')] ]
13 });
14 // Ausführen des nativen codes
15 var greeter = ref.alloc('void*');
16 lib.PROTO_New(greeter);
17 lib.PROTO_Greet(greeter, "ESD");
18 lib.PROTO_Delete(greeter);
```

Das Beispiel 8.3 öffnet je nach Plattform die entsprechende Programmbibliothek, lädt die darin exportierten Funktionen und ruft sie auf. Dieser Code ist mit Hilfe der NodeJS Javascript-Laufzeitumgebung ausführbar:

Listing 8.4: Ausgabe des Tests.

```
1 $ nodejs ./node.js
2 Hello, ESD
```

In der Ausgabe 8.4 ist erkennbar, dass der native Code erfolgreich ausgeführt wird.

Der auf der CD-ROM/DVD unter *Thesis/prototype4* beigelegte Beispielcode kann mit folgenden Anweisungen übersetzt werden:

Linux und Windows:

```
1 $ mkdir build
2 $ cd ./build
3 $ cmake ..
4 $ cmake --build .
```

8.2.2 Fazit

Aufgrund dieser Erkenntnisse wird zukünftig innerhalb der Abteilung *DI FA SO SIS* der Fokus auf diese FFI-Technologie gelegt. Die Idee ist es, die C++-Klassen der WinCC OA Programmierschnittstelle nach der vorher gezeigten Methodik für andere Programmiersprachen zur Verfügung zu stellen. Daraus würde sich der weitere Vorteil ergeben, dass diese so erstellte Programmbibliothek nicht mehr an die Compilervorgaben des WinCC OA Produktes gebunden ist. Ein funktionaler konzeptioneller Prototyp mit Beispiel Marshallingcode für Javascript (NodeJS), Python und C# (.NET Core) ist der CD-ROM/DVD unter *corelib* beigelegt.

Anhang A

Google V8 Javascriptengine Buildskripte

A.1 Linux

```
#!/bin/bash
# Author: Daniel Giritzer
# This script builds google V8 from source.
# To be used on Ubuntu 18.04
# to install and setup dependencies execute:
# sudo apt install git libglib2.0-dev python g++-4.8 gcc-4.8
# sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-4.8 10
# sudo update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-4.8 10
WORKDIR=$(pwd)
OUTDIR=$WORKDIR/v8
BUILDDIR=$WORKDIR/v8/out/v8
ARTEFACTDIR=$WORKDIR/lib/sles_x64-release
INCLUDEDIR=$WORKDIR/include
VERSIONTAG="6.2.414.5"

# clean
rm -rf $WORKDIR/.cipd
rm -rf $WORKDIR/.gclient
rm -rf $WORKDIR/.gclient_entries
rm -rf $WORKDIR/depot_tools
rm -rf $WORKDIR/v8

# get depot tools
git clone https://chromium.googlesource.com/chromium/tools/depot_tools.git

# add depot tools to path
OLDPATH=$PATH
export PATH=$PATH:$WORKDIR/depot_tools/

# get v8 source
fetch v8
cd $OUTDIR && git checkout $VERSIONTAG && gclient sync -j 4

# extract archive and create needed folders
mkdir -p $BUILDDIR
mkdir -p $ARTEFACTDIR

```

```
38 # set path to find build executables remove gn wrapper (not needed)
39 export PATH=$OLDPATH:$OUTDIR/third_party/depot_tools/:$OUTDIR/buildtools/linux64/
40 rm -rf $OUTDIR/third_party/depot_tools/gn
41
42 # build configuration
43 echo 'is_component_build = false' >> $BUILDDIR/args.gn
44 echo 'is_debug = false' >> $BUILDDIR/args.gn
45 echo 'target_cpu = "x64"' >> $BUILDDIR/args.gn
46 echo 'use_goma = false' >> $BUILDDIR/args.gn
47 echo 'goma_dir = "None"' >> $BUILDDIR/args.gn
48 echo 'v8_enable_backtrace = true' >> $BUILDDIR/args.gn
49 echo 'v8_enable_disassembler = true' >> $BUILDDIR/args.gn
50 echo 'v8_enable_object_print = true' >> $BUILDDIR/args.gn
51 echo 'v8_enable_verify_heap = true' >> $BUILDDIR/args.gn
52 echo 'v8_static_library = true' >> $BUILDDIR/args.gn
53 echo 'v8_use_external_startup_data = false' >> $BUILDDIR/args.gn
54 echo 'v8_use_snapshot = true' >> $BUILDDIR/args.gn
55 echo 'is_clang = false' >> $BUILDDIR/args.gn
56 echo 'use_sysroot = false' >> $BUILDDIR/args.gn
57
58 # generate build files
59 cd $OUTDIR && gn gen $BUILDDIR
60
61 # workaround to prevent build from failing with g++ 4.8.5, force use of C++11
62 find $BUILDDIR -type f -exec sed -i 's/-Werror//g' {} \;
63 find $BUILDDIR -type f -exec sed -i 's/-std=gnu++14/-std=gnu++11 -fPIC -
       D_GLIBCXX_USE_CXX11_ABI=0/g' {} \;
64
65 # start build with verbose output
66 ninja -v -j 4 -C $BUILDDIR d8
67
68 # copy resulting librariers
69 find $BUILDDIR -name *.a -exec cp {} $ARTEFACTDIR \;
70
71 # copy include directory
72 mkdir -p $INCLUDEDIR
73 cp -rf $OUTDIR/include/* $INCLUDEDIR
74 cp -rf $BUILDDIR/gen/include/inspector/ $INCLUDEDIR
```

A.2 Windows

```
 1 # Author: Daniel Giritzer
 2 # This script builds google V8 from source.
 3 # Execute from cmd via:
 4 # powershell.exe -noprofile -executionpolicy bypass -file .\build_windows.ps1
 5 $WORKDIR="$pwd"
 6 $OUTDIR="$WORKDIR\v8"
 7 $BUILDDIR="$WORKDIR\v8\out\v8"
 8 $ARTEFACTDIR="$WORKDIR\lib\win-x64-release"
 9 $INCLUDEDIR="$WORKDIR\include"
10 $VERSIONTAG="6.2.414.5"
11
12 # helper function for unpacking zip archives
13 Add-Type -AssemblyName System.IO.Compression.FileSystem
14 function Unzip
15 {
```

```
16     param([string]$zipfile, [string]$outpath)
17     [System.IO.Compression.ZipFile]::ExtractToDirectory($zipfile, $outpath)
18 }
19
20 # download depot_tools
21 $CLIENT = new-object System.Net.WebClient
22 $CLIENT.DownloadFile("https://storage.googleapis.com/chrome-infra/depot_tools.zip","
       depot_tools.zip")
23 Unzip "depot_tools.zip" "depot_tools"
24
25 # add depot tools to path
26 $env:DEPOT_TOOLS_WIN_TOOLCHAIN=0
27 $env:GYP_MSVS_VERSION=2015
28 $OLDPATH=$env:Path
29 $env:Path="$pwd\depot_tools;$OLDPATH"
30
31 # get files needed for windows build
32 gclient
33
34 # get v8 sourcecode
35 fetch v8
36 cd "$OUTDIR"
37 git checkout $VERSIONTAG
38 gclient sync
39
40 New-Item -ItemType directory -Path $BUILDDIR
41
42 # build configuration
43 Add-Content $BUILDDIR\args.gn 'is_component_build = false'
44 Add-Content $BUILDDIR\args.gn 'is_debug = false'
45 Add-Content $BUILDDIR\args.gn 'target_cpu = "x64"'
46 Add-Content $BUILDDIR\args.gn 'use_goma = false'
47 Add-Content $BUILDDIR\args.gn 'goma_dir = "None"'
48 Add-Content $BUILDDIR\args.gn 'v8_enable_backtrace = true'
49 Add-Content $BUILDDIR\args.gn 'v8_enable_disassembler = true'
50 Add-Content $BUILDDIR\args.gn 'v8_enable_object_print = true'
51 Add-Content $BUILDDIR\args.gn 'v8_enable_verify_heap = true'
52 Add-Content $BUILDDIR\args.gn 'v8_static_library = true'
53 Add-Content $BUILDDIR\args.gn 'v8_use_external_startup_data = false'
54 Add-Content $BUILDDIR\args.gn 'v8_use_snapshot = true'
55 Add-Content $BUILDDIR\args.gn 'is_clang = false'
56
57 # workaround to link using /MD
58 (Get-Content $OUTDIR\build\config\win\BUILD.gn).replace('/MT', '/MD') | Set-Content
       $OUTDIR\build\config\win\BUILD.gn
59
60 # generate build files
61 cd $OUTDIR
62 gn gen $BUILDDIR
63
64 # start build with verbose output
65 ninja -v -C $BUILDDIR d8
66
67 # copy artefacts
68 Get-ChildItem -Path "$BUILDDIR\*" -Include *.lib -Recurse | Copy-Item -Destination
       $ARTEFACTDIR
69
```

```
70 # copy include directory
71 Remove-Item -Recurse -Force $INCLUDEDIR
72 Copy-Item $OUTDIR\include -Recurse -Destination $INCLUDEDIR
73 Copy-Item $OUTDIR\gen\include -Recurse -Destination $INCLUDEDIR
```

Anhang B

Inhalt der CD-ROM/DVD

Die CD-ROM/DVD beinhaltet den *SiPlant* Quellcode in der Version 2.1 und diese Arbeit in *LaTeX*-Form.

Der Inhalt dieser CD-ROM/DVD gliedert sich in folgende Verzeichnisse:

- **Thesis:** In dem Verzeichnis Thesis ist der *LaTeX* Quellcode dieser Arbeit inklusive Versionskontrolle und Beispielprogrammen abgelegt. Das verwendete Versionskontrollsystem ist *git* [35].
- **corelib:** WinCC OA als Fremdfunktionsschnittstelle, ein konzeptioneller Beweiß.
- **SMF_3P:** Hier sind folgende externen Komponenten mit Quellcode, Kompilat und Skripte zum Kompilieren abgelegt, die zum Übersetzten des im SMF_Native Verzeichnis abgelegten Quellcodes benötigt werden: *ACE* [46], *boost* [34], *GTEngine*, *mbedtls* [45], *OpenSSL* [25], *SQLite* [26], *Google V8 Engine* [36] und *WebsocketPP* [24].
- **SMF_Native:** Dieser Ordner beinhaltet den C++-Quellcode von *SiPlant* in der Version 2.1.
- **SMF_Common:** Subprojekt mit Sammlungen von Sprachübersetzungsdateien, Farbdefinitionen und Datenpunktlisten (Version 2.1).
- **SMF_PlantVisu:** Erweiterndes Modul mit verschiedenen Auswertungen für die Anlagenvisualisierung (Version 2.1).
- **SMF_SmartScheduler:** In diesem Verzeichnis befindet sich die Schichtkalender-Erweiterung (Version 2.1).

Anhang C

Ausgabe einer V8-Javascript-Exception

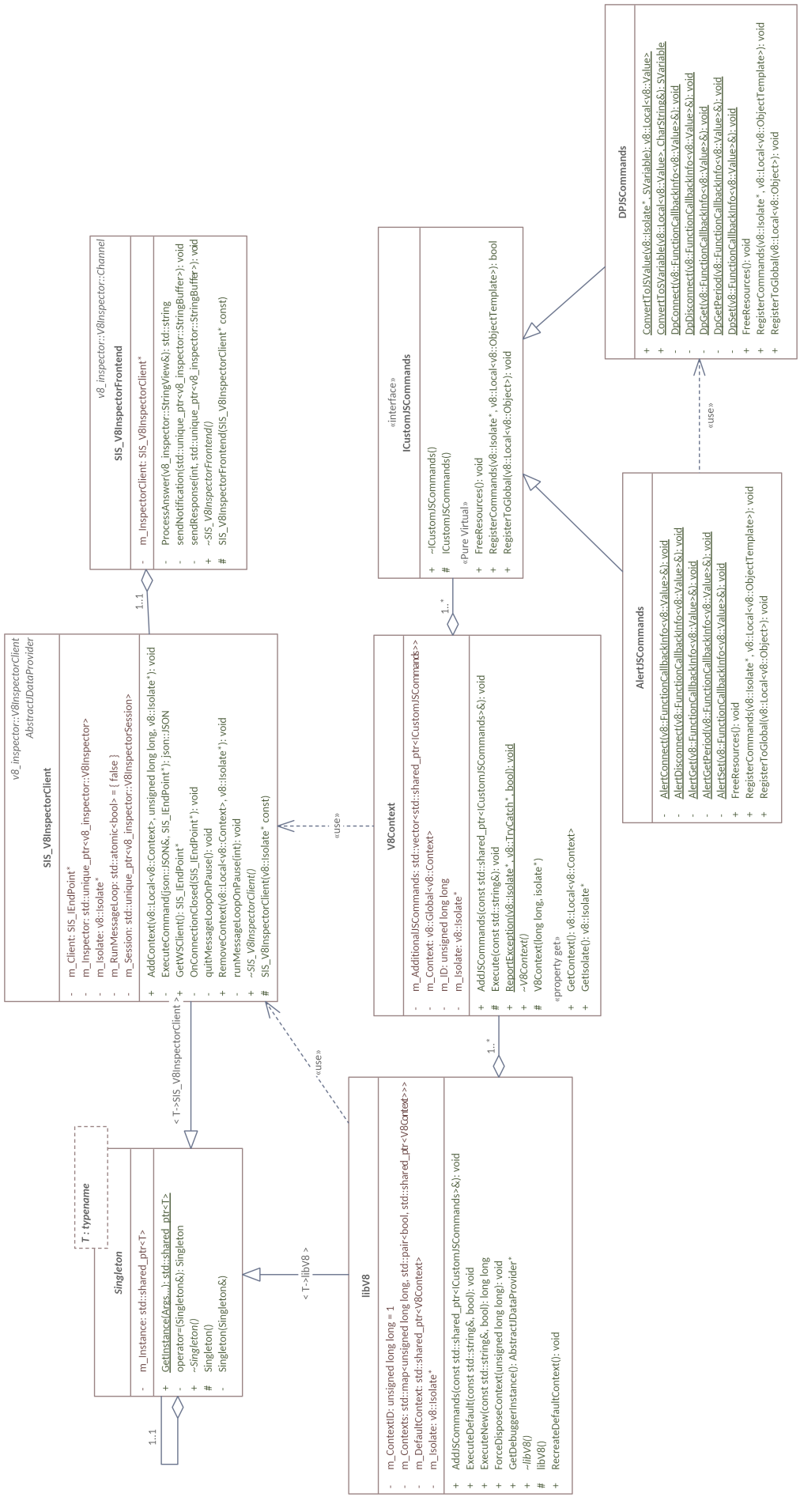
Listing C.1: Gibt den Inhalt eines V8-*TryCatch*-Objekts aus.

```
0 void ReportException(v8::Isolate* isolate, v8::TryCatch* try_catch)
1 {
2   v8::HandleScope handle_scope(isolate);
3   v8::String::Utf8Value exceptionV8(try_catch->Exception());
4   std::string exception = *exceptionV8;
5   v8::Local<v8::Message> message = try_catch->Message();
6   // Gebe nur Exception aus, falls keine weiteren Infos vorhanden
7   if (message.IsEmpty()) {
8     std::cout << exception << std::endl;
9   }
10  else {
11    // Dateiname
12    v8::String::Utf8Value filenameV8(message->GetScriptOrigin().ResourceName());
13    std::string fileName = *filenameV8;
14    // Zeilennummer
15    v8::Local<v8::Context> context(isolate->GetCurrentContext());
16    int linenum = message->GetLineNumber(context).FromJust();
17    // Zeileninhalt
18    v8::String::Utf8Value sourcelineV8(message->GetSourceLine(context).
      ToLocalChecked());
19    std::string sourceLine = *sourcelineV8;
20    // Schreibe Dateinamen, Zeilennummer und Zeileninhalt
21    std::cout << fileName << ":" << linenum << std::endl;
22    std::cout << sourceLine << std::endl;
23    // Schreibe Stacktrace, falls vorhanden
24    v8::Local<v8::Value> stackTraceV8;
25    if (try_catch->StackTrace(context).ToLocal(&stackTraceV8) &&
26        stackTraceV8->IsString() &&
27        v8::Local<v8::String>::Cast(stackTraceV8)->Length() > 0) {
28      v8::String::Utf8Value stackTraceV8String(stackTraceV8);
29      std::string stackTrace = *stackTraceV8String;
30      std::cout << stackTrace << std::endl;
31    }
32  }
33 }
```

Anhang D

Klassendiagramm

Siehe nächste Seite.


v8_inspector::V8InspectorClient
AbstractIDataProvider
SIS_V8InspectorClient
- m_Client: SIS_IEndPoint*
- m_Inspector: std::unique_ptr<v8_inspector::V8Inspector>
- m_Isolate: v8::Isolate*
- m_RunMessageLoop: std::atomic<bool> = { false }
- m_Session: std::unique_ptr<v8_inspector::V8InspectorSession>
+ AddContext(v8::Local<v8::Context>, unsigned long long, v8::Isolate*): void
- ExecuteCommand(json::JSON&, SIS_IEndPoint*): json::JSON
+ GetWSClient(): SIS_IEndPoint*
- OnConnectionClosed(SIS_IEndPoint*): void
- quitMessageLoopOnPause(): void
+ RemoveContext(v8::Local<v8::Context>, v8::Isolate*): void
- runMessageLoopOnPause(int): void
+ ~SIS_V8InspectorClient()
SIS_V8InspectorClient(v8::Isolate* const)
v8_inspector::V8Inspector::Channel
SIS_V8InspectorFrontend
- m_InspectorClient: SIS_V8InspectorClient*
- ProcessAnswer(v8_inspector::StringView&): std::string
- sendNotification(std::unique_ptr<v8_inspector::StringBuffer>): void
- sendResponse(int, std::unique_ptr<v8_inspector::StringBuffer>): void
+ ~SIS_V8InspectorFrontend()
SIS_V8InspectorFrontend(SIS_V8InspectorClient* const)
1..1
«interface»
ICustomJSCommands
+ ~ICustomJSCommands()
ICustomJSCommands()
«Pure Virtual»
+ FreeResources(): void
+ RegisterCommands(v8::Isolate* , v8::Local<v8::ObjectTemplate>): bool
+ RegisterToGlobal(v8::Local<v8::Object>): void
DPJSCommands
+ ConvertToJSValue(v8::Isolate* ,SVariable): v8::Local<v8::Value>
+ ConvertToSVariable(v8::Local<v8::Value>, CharString&): SVariable
- DpConnect(v8::FunctionCallbackInfo<v8::Value>&): void
- DpDisconnect(v8::FunctionCallbackInfo<v8::Value>&): void
- DpGet(v8::FunctionCallbackInfo<v8::Value>&): void
- DpGetPeriod(v8::FunctionCallbackInfo<v8::Value>&): void
- DpSet(v8::FunctionCallbackInfo<v8::Value>&): void
+ FreeResources(): void
+ RegisterCommands(v8::Isolate* , v8::Local<v8::ObjectTemplate>): void
+ RegisterToGlobal(v8::Local<v8::Object>): void
«use»
AlertJSCommands
- AlertConnect(v8::FunctionCallbackInfo<v8::Value>&): void
- AlertDisconnect(v8::FunctionCallbackInfo<v8::Value>&): void
- AlertGet(v8::FunctionCallbackInfo<v8::Value>&): void
- AlertGetPeriod(v8::FunctionCallbackInfo<v8::Value>&): void
- AlertSet(v8::FunctionCallbackInfo<v8::Value>&): void
+ FreeResources(): void
+ RegisterCommands(v8::Isolate* , v8::Local<v8::ObjectTemplate>): void
+ RegisterToGlobal(v8::Local<v8::Object>): void
1..*
V8Context
- m_AdditionalJSCommands: std::vector<std::shared_ptr<ICustomJSCommands>>
- m_Context: v8::Global<v8::Context>
- m_ID: unsigned long long
- m_Isolate: v8::Isolate*
+ AddJSCommands(const std::shared_ptr<ICustomJSCommands>&): void
Execute(const std::string&): void
+ ReportException(v8::Isolate* ,v8::TryCatch*, bool): void
+ ~V8Context()
V8Context(long long, isolate*)
«property get»
+ GetContext(): v8::Local<v8::Context>
+ GetIsolate(): v8::Isolate*
«use»
< T->SIS_V8InspectorClient >
T : typename
Singleton
- m_Instance: std::shared_ptr<T>
+ GetInstance(Args...): std::shared_ptr<T>
- operator=(Singleton&): Singleton
+ ~Singleton()
Singleton()
- Singleton(Singleton&)
1..1
«use»
< T->libV8 >
libV8
- m_ContextID: unsigned long long = 1
- m_Contexts: std::map<unsigned long long, std::pair<bool, std::shared_ptr<V8Context>>>
- m_DefaultContext: std::shared_ptr<V8Context>
- m_Isolate: v8::Isolate*
+ AddJSCommands(const std::shared_ptr<ICustomJSCommands>&): void
+ ExecuteDefault(const std::string&, bool): void
+ ExecuteNew(const std::string&, bool): long long
+ ForceDisposeContext(unsigned long long): void
+ GetDebuggerInstance(): AbstractIDataProvider*
+ ~libV8()
libV8()
+ RecreateDefaultContext(): void

Abbildungsverzeichnis

1.1 Diese Abbildung stellt die verschiedenen Ebenen der industriellen Fertigung mit Hilfe der Automatisierungspyramide dar [21]. Cloudanwendungen befinden sich in der Ebene vier, die auch *Unternehmensleitebene* genannt wird. 1

2.1 Ein WinCC OA System besteht aus verschiedenen funktionsspezifischen Agenten zusammen. 7
2.2 Diese Abbildung zeigt die grafischen Bedienelemente zum Modellieren von Datenpunkttypen und Anlegen von Datenpunktinstanzen. 8
2.3 Ein redundant konfiguriertes System. 10
2.4 Zwei verteilte Systeme. 11
2.5 Zusammenspiel zwischen Manager-Klasse und *HotLinkWaitForAnswer*-Interface. 15
2.6 SIS_HTTP, SIS_Event und SIS_NGUI als Erweiterung für WinCC OA. 19

3.1 Diese Abbildung zeigt die Sprachprozessoren Compiler und Interpreter im Vergleich. 20
3.2 Schritte des Frontends [20]. 21
3.3 Abstrakter Syntaxbaum des Beispielcodes. 25
3.4 In dieser Illustration wird die Unabhängigkeit der verschiedenen Contexts zueinander veranschaulicht [22]. 28
3.5 Speicherverwaltung mit Hilfe von Handles.[23] 29

4.1 Dieses Ablaufdiagramm stellt die Entwicklung kundenspezifischer Anforderungen grafisch dar. 36
4.2 In diesem Diagramm wird die Massenparametrierung dargestellt. 37

5.1 Dieses Übersichtsbild zeigt die statische Bibliothek *libV8* und wie sie verwendet wird. 42
5.2 Generierte Dokumentation für *DpGet*. 43

6.1 Singleton Template-Klasse. 49
6.2 libV8-Singleton-Klasse. 50
6.3 SIS_InspectorClient-Klasse. 51
6.4 Implementierung des *v8Inspector::Channel*-Interfaces. 52
6.5 V8Context-Klasse, repäsentiert einen Ausführungskontext. 52

6.6 ICustomJSCommands-Interface zum Erweitern des Javascript-Sprachumfangs. 53
6.7 Funktionen für den Datenbankzugriff. 54
6.8 Funktionen für den Alarmzugriff. 54

D.1 Klassendiagramm der implementierten Programmkomponente. 69

Quellenverzeichnis

Literatur

[1] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, Aug. 2006 (siehe S. 20).

[2] Sang Hyuk Son Azer Bestavros Kwei-Jay Lin, Hrsg. *Real-Time Database Systems*. Springer US, 1997 (siehe S. 5).

[3] L. Thomas van Binsbergen u. a. „A Principled Approach to REPL Interpreters“. In: *Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. Onward! 2020. Virtual, USA: Association for Computing Machinery, 2020, S. 84–100. URL: https://doi.org/10.1145/3426428.3426917 (siehe S. 30).

[4] Constantin Szallies. *On using the Oberver Design Pattern*. Aug. 1997 (siehe S. 14).

[5] ETM Professional Control. *WinCC Open Architecture 3.16 Documentation*. 2019 (siehe S. 2, 7).

[6] George Coulouris, Jean Dollimore und Tim Kindberg. *Distributed Systems: Concepts and Design (International Computer Science)*. 4th rev. ed. Addison-Wesley Longman, Amsterdam, 2005 (siehe S. 45).

[7] Marcus Crestani. „Foreign-Function Interfaces for Garbage-Collected Programming Languages“ (2008) (siehe S. 59).

[8] A. Daneels und W. Salter. „What is SCADA?“ In: 1999 (siehe S. 2, 5).

[9] Dimitri van Heesch. *doxygen, Manual for version 1.8.13*. 2016. URL: http://doxygen.nl/download.html#latestman (siehe S. 42).

[10] Hans-Arno Jacobsen. „Publish/Subscribe“. In: *Encyclopedia of Database Systems*. Hrsg. von LING LIU und M. TAMER ÖZSU. Boston, MA: Springer US, 2009, S. 2208–2211. URL: https://doi.org/10.1007/978-0-387-39940-9_1181 (siehe S. 5).

[11] Michael K. Johnson. *Anwendungen entwickeln unter Linux*. Bonn Reading, Mass. u.a: Addison-Wesley-Longman, 1998. URL: http://www.addison-wesley.de/3827314496.html (siehe S. 40).

[12] Donald E. Knuth. „Literate Programming“. *Comput. J.* 27.2 (Mai 1984), S. 97–111. URL: http://dx.doi.org/10.1093/comjnl/27.2.97 (siehe S. 42).

[13] Stuart Maclean. *On the Singleton Software Design Pattern*. Techn. Ber. United Kingdom, Highfield, Southampton SO17: University of Southampton, Department of Electronics und Computer Science, 1997 (siehe S. 13, 49).

[14] Martin Polke. *Prozessleittechnik*. 2. Aufl. Polke, Martin, 1994 (siehe S. 2).

[15] Steven Pritchard u. a. *LPI Linux Certification in a Nutshell (In a Nutshell (O'Reilly))*. O'Reilly Media, Inc., 2006 (siehe S. 41).

[16] *SpiderMonkey Developer Documentation*. 2021. URL: https://firefox-source-docs.mozilla.org/js/index.html (besucht am 04. 08. 2021) (siehe S. 22).

[17] Michael Wooldringe Stefan Bussmann Nicholas R. Jennings. *Multiagent Systems For Manufactoring Control*. 2. Aufl. Springer, 2004 (siehe S. 6).

[18] P. Terry. „Compilers and compiler generators : an introduction with C++". In: 1997 (siehe S. 20).

[19] Michael Wooldridge und Nicholas R. Jennings. „Intelligent agents: theory and practice". *The Knowledge Engineering Review* 10.2 (1995), S. 115–152 (siehe S. 6, 7).

Audiovisuelle Medien

[20] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Hardcover. Compiler-Frontend. Aug. 2006 (siehe S. 21).

[21] Martin Polke. *Prozessleittechnik*. Automatisierungspyramide. 1994 (siehe S. 1).

[22] *V8 Embedding Documentation*. Funktionsweise Contexts. 2019. URL: https://v8.dev/docs/embed (besucht am 15. 12. 2019) (siehe S. 28).

[23] *V8 Embedding Documentation*. Handles und Garbagecollection. 2019. URL: https://v8.dev/docs/embed (besucht am 15. 12. 2019) (siehe S. 29).

Online-Quellen

[24] *A C++ library that can be used to implement WebSocket functionality*. 2021. URL: https://docs.websocketpp.org/ (besucht am 02. 08. 2021) (siehe S. 66).

[25] *A robust, commercial-grade, and full-featured toolkit for the Transport Layer Security (TLS) and Secure Sockets Layer (SSL) protocols*. 2021. URL: https://www.openssl.org/ (besucht am 02. 08. 2021) (siehe S. 66).

[26] *A small, fast, self-contained, high-reliability, full-featured, SQL database engine*. 2021. URL: https://www.sqlite.org/index.html (besucht am 02. 08. 2021) (siehe S. 66).

[27] Daniel Bevenius. *Learning V8*. 2021. URL: https://github.com/danbev/learning-v8 (besucht am 16. 08. 2021) (siehe S. 27).

[28] *Browser Market Share, Desktop and Mobile*. 2021. URL: https://netmarketshare.com/browser-market-share.aspx (besucht am 17. 08. 2021) (siehe S. 23).

[29] *CentOS Website*. 2021. URL: https://www.centos.org/ (besucht am 17. 08. 2021) (siehe S. 35).

[30] *CMake Homepage*. 2021. URL: https://cmake.org/ (besucht am 16. 08. 2021) (siehe S. 18, 33, 47).

[31] *DebugInspector Protocoll*. 2019. URL: https://chromedevtools.github.io/debugger-protocol-viewer/tot/ (besucht am 15. 12. 2019) (siehe S. 30).

[32] *Desktop and Mobile Browser Market Share Worldwide*. 2021. URL: https://gs.statcounter.com/browser-market-share/desktop-mobile/worldwide/#monthly-202107-202107-bar (besucht am 17. 08. 2021) (siehe S. 23).

[33] *ECMA Standarts*. 2021. URL: https://www.ecma-international.org/publications-and-standards/standards/ (besucht am 05. 08. 2021) (siehe S. 20, 57).

[34] *Free peer-reviewed portable C++ source libraries.* 2021. URL: https://www.boost.org/ (besucht am 02. 08. 2021) (siehe S. 66).

[35] *Git Homepage*. 2021. URL: https://git-scm.com/ (besucht am 02. 08. 2021) (siehe S. 66).

[36] *Google's open source high-performance JavaScript and WebAssembly engine, written in C++*. 2021. URL: https://v8.dev/ (besucht am 02. 08. 2021) (siehe S. 66).

[37] Franziska Hinkelmann. *Understanding V8's Bytecode*. 2017. URL: https://fhinkel.rocks/2017/08/16/Understanding-V8-s-Bytecode/ (besucht am 04. 08. 2021) (siehe S. 22).

[38] *How (not) to access V8 memory*. 2019. URL: https://nodeaddons.com/how-not-to-access-node-js-from-c-worker-threads/ (besucht am 15. 12. 2019) (siehe S. 27).

[39] *Javascript Web APIs*. 2021. URL: https://www.w3.org/standards/webdesign/script.html (besucht am 05. 08. 2021) (siehe S. 57).

[40] *Launching Ignition and TurboFan*. 2021. URL: https://docs.google.com/document/d/11T2CRex9hXxoJwbYqVQ32yIPMh0uouUZLdyrtmMoL44/mobilebasic#h.z0c1oogsbnnq (besucht am 16. 08. 2021) (siehe S. 24).

[41] Ross McIlroy. *Ignition: V8 Interpreter*. 2021. URL: https://docs.google.com/document/d/11T2CRex9hXxoJwbYqVQ32yIPMh0uouUZLdyrtmMoL44/mobilebasic#h.z0c1oogsbnnq (besucht am 16. 08. 2021) (siehe S. 24).

[42] Benedikt Meurer. *An Introduction to Speculative Optimization in V8*. 2021. URL: https://ponyfoo.com/articles/an-introduction-to-speculative-optimization-in-v8 (besucht am 17. 08. 2021) (siehe S. 24).

[43] *NodeJS Website*. 2021. URL: https://nodejs.org/en/ (besucht am 17. 08. 2021) (siehe S. 23, 59).

[44] Assaf Sion. *The Mysterious Realm of JavaScriptCore*. 2021. URL: https://www.cyberark.com/resources/threat-research-blog/the-mysterious-realm-of-javascriptcore (besucht am 04. 08. 2021) (siehe S. 22).

[45] *SSL library with an intuitive API and readable source code.* 2021. URL: https://tls.mbed.org/ (besucht am 02. 08. 2021) (siehe S. 66).

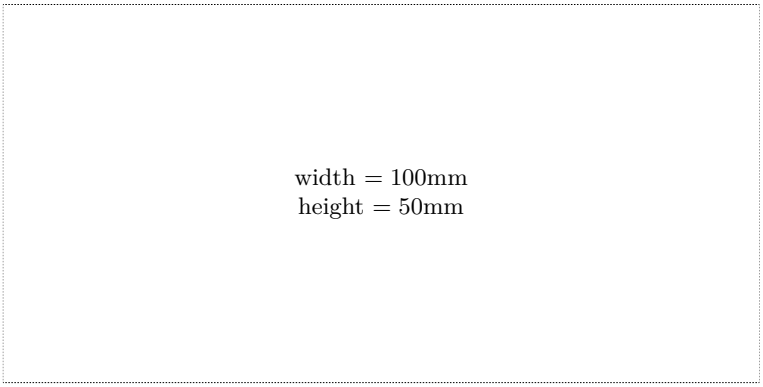
[46] *The ADAPTIVE Communication Environment (ACE(TM))*. 2021. URL: http://www.dre.vanderbilt.edu/~schmidt/ACE.html (besucht am 02. 08. 2021) (siehe S. 66).

[47] *TypeDoc Homepage*. 2021. URL: https://typedoc.org/ (besucht am 21. 08. 2021) (siehe S. 42).

[48] *V8 Embedding Documentation*. 2019. URL: https://v8.dev/docs/embed (besucht am 15. 12. 2019) (siehe S. 27).

Messbox zur Druckkontrolle

— Druckgröße kontrollieren! —



— Diese Seite nach dem Druck entfernen! —